ANSIBLE – CHALLENGE

Create ci pipeline that will deploy and configure these VMs using terraform and ansible according following requirements

1. Deploy 2 virtual machines using terraform.
    - first vm on Amazon linux, hostname: c8.local
    - second vm on ubuntu 21.04, hostname: u21.local
2. As a result of terraform execution, dynamically create inventory for ansible
    - c8.local should be in the frontend group
    - u21.local should be in the backend group
3. Create ansible playbook for c8.local and u21.local

- for linux OS playbook should apply the following changes
    - selinux: disable
    - firewalld: disable
- for frontend playbook group should install and configure nginx
    - nginx configuration should do proxying from port 80 on port 19999 to the
    - backend group
- for the backend group, the playbook must install the Netdata application from the
    - official repositories and run it on port 19999.

CI Pipeline: Terraform + Ansible + Nginx + Netdata on AWS

We'll build an end-to-end CI pipeline that provisions two AWS EC2 virtual machines using Terraform, dynamically generates an Ansible inventory, configures both Linux servers, and deploys Nginx as a reverse proxy to Netdata.

The examples below use your existing AWS/Ansible lab experience and Git Bash or an EC2-based Ansible control node.

1. Target architecture

CI Pipeline — Jenkins

Checkout → Terraform → Dynamic inventory → Ansible

AWS VPC

Frontend VM

c8.local

Amazon Linux

Nginx

Port 80 → 19999

Backend VM

u21.local

Ubuntu 21.04*

Netdata

Port 19999

Nginx proxies requests to the backend VM's private IP on port 19999.

Important OS note: Ubuntu 21.04 is end-of-life and no longer receives normal security updates. For a real production deployment, use a supported Ubuntu LTS release, such as 24.04. If Ubuntu 21.04 is mandatory for your lab, you must source its old image carefully and may need to use archived package repositories. Similarly, c8.local is a requested hostname, not an automatically registered public DNS name.

2. How the CI pipeline will work

Stage	Action
1. Checkout	Get Terraform and Ansible files from Git
2. Validate	Run terraform fmt and terraform validate
3. Provision	Create both EC2 instances and networking

4. Inventory	Generate ansible/inventory.ini from Terraform outputs
5. Configure OS	Disable SELinux where supported and stop/disable firewalld where installed
6. Configure backend	Install and start Netdata on port 19999
7. Configure frontend	Install Nginx and configure reverse proxy
8. Verify	Test Nginx on port 80 and Netdata on port 19999

Networking principle: Nginx will connect to the backend using its private IP address. The backend security group should allow port 19999 only from the frontend security group, not from the whole internet.

3. Prerequisites

You need the following on your Jenkins agent or Ansible control node:

- AWS account and credentials with permissions to create EC2 instances, security groups, and networking resources.
- Terraform.
- Ansible.
- AWS CLI.
- A Git repository containing the pipeline files.
- An EC2 SSH key pair whose private key is available securely to the pipeline.

Since you've used Jenkins on EC2 before, I recommend running the pipeline on a dedicated Jenkins agent with Terraform, Ansible, AWS CLI, and Git installed.

Check the tools:

terraform -version

ansible --version

aws --version

git --version

Check AWS authentication:

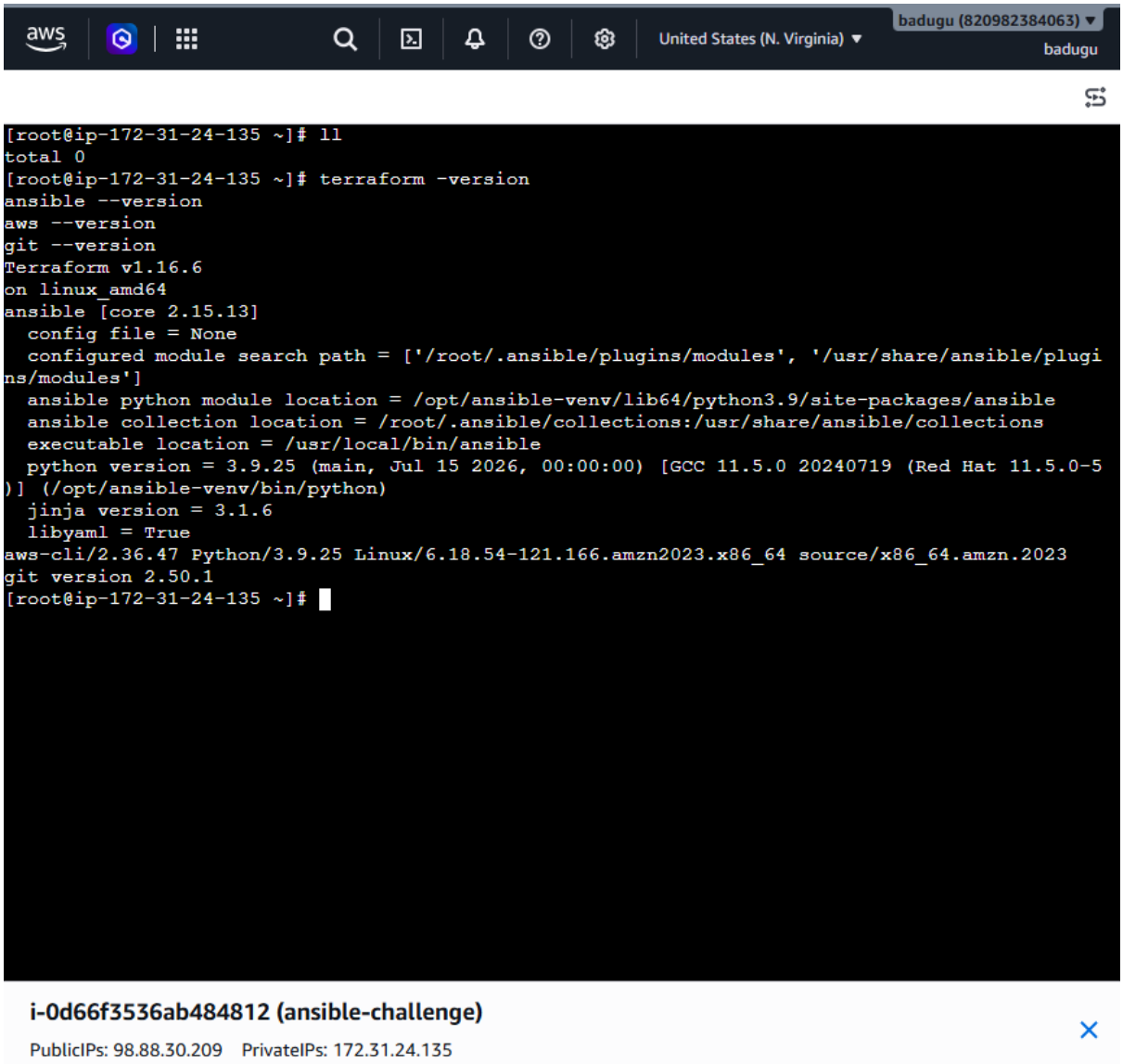
aws sts get-caller-identity

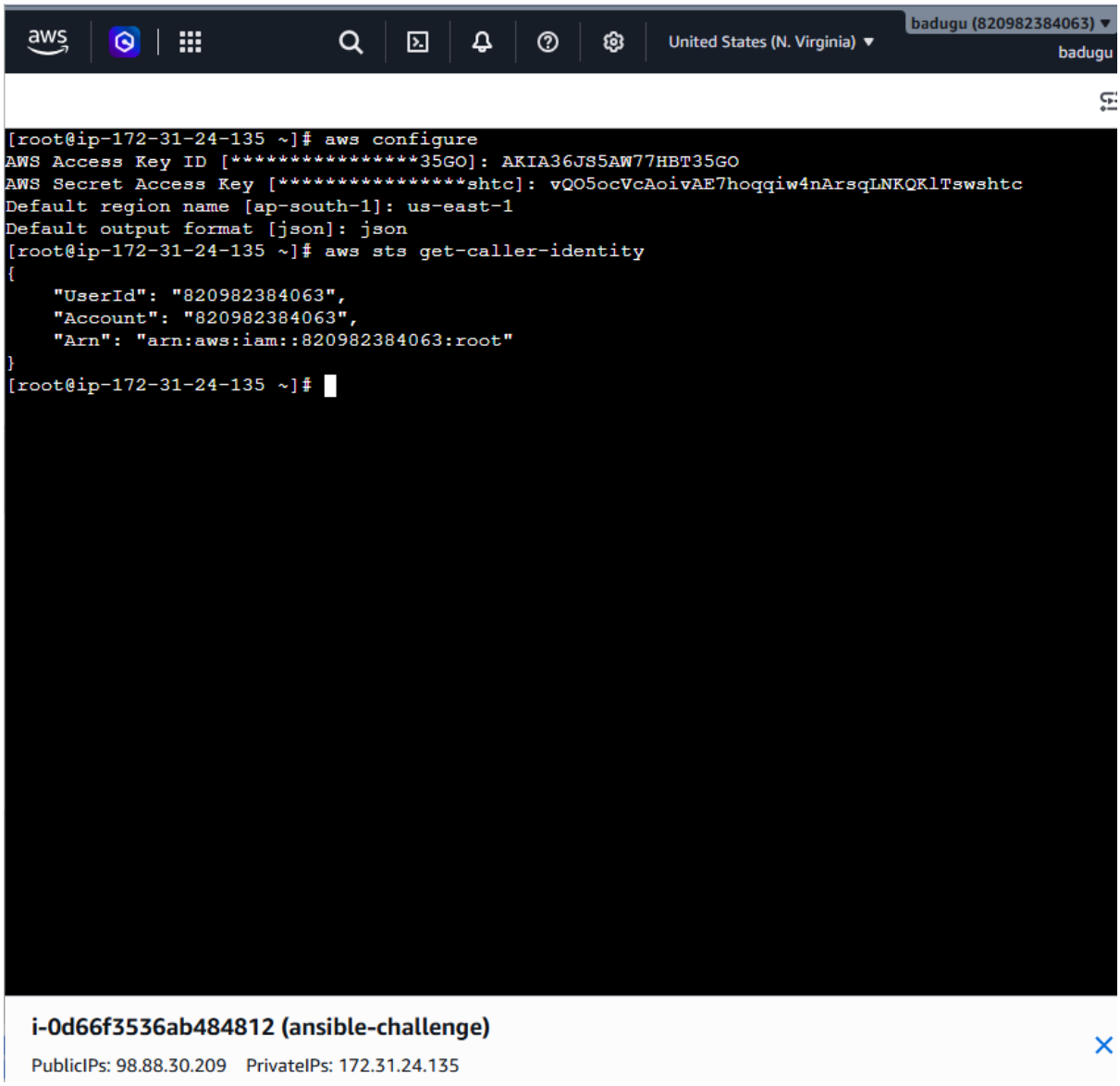
aws configure get region

Set the region if required:

aws configure set region us-east-1

Do not put AWS access keys or private SSH keys in Git. Use a Jenkins AWS credential/role and a Jenkins-managed SSH private-key credential.





4. Create the project directory

Create the following structure in your Git repository:

terraform-ansible-ci/

├── Jenkinsfile

├── terraform/

│ ├── main.tf

│ ├── variables.tf

│ ├── outputs.tf

```
│   └── terraform.tfvars.example
└── ansible/
    ├── site.yml
    └── ansible.cfg
```

Create the directories:

```
mkdir -p terraform-ansible-ci/{terraform,ansible}
cd terraform-ansible-ci
```

4. Create the project directory

Create the following structure in your Git repository:

```
terraform-ansible-ci/
├── Jenkinsfile
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── terraform.tfvars.example
└── ansible/
    ├── site.yml
    └── ansible.cfg
```

Create the directories:

```
mkdir -p terraform-ansible-ci/{terraform,ansible}
cd terraform-ansible-ci
```

The Terraform configuration will create the two EC2 instances and generate the Ansible inventory automatically.

5. Terraform configuration

We'll use the default VPC and its subnets for a simple lab. The configuration creates separate security groups for the frontend and backend.

5.1 Create terraform/variables.tf

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "key_name" {
  description = "Existing AWS EC2 key pair name"
  type        = string
}

variable "ssh_private_key_path" {
  description = "Path to the private SSH key on the Terraform runner"
  type        = string
}

variable "ssh_allowed_cidr" {
  description = "Your public IP in CIDR format, e.g. 203.0.113.10/32"
  type        = string
}
```

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "ubuntu_2104_ami_id" {
  description = "Existing Ubuntu 21.04 AMI ID in the selected region"
  type        = string
}

variable "ubuntu_2104_username" {
  description = "SSH username for the selected Ubuntu AMI"
  type        = string
  default     = "ubuntu"
}
```

Important: Ubuntu 21.04 is obsolete. You must supply an existing compatible AMI ID, if one is still available in your AWS region. Do not assume a historical AMI ID will still launch. If you can use Ubuntu 24.04 LTS instead, I can simplify the AMI selection considerably.

5.2 Create terraform/main.tf

This configuration uses the current Amazon Linux 2023 AMI published through AWS Systems Manager Parameter Store and your supplied Ubuntu AMI ID.

```
terraform {
  required_version = ">= 1.6.0"
```

```
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }

    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
```

```
}

data "aws_ssm_parameter" "amazon_linux_2023" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

resource "aws_security_group" "frontend" {
  name_prefix = "ci-frontend-"
  description = "Frontend HTTP and restricted SSH"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "SSH from administrator"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_allowed_cidr]
  }
```

```
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "ci-frontend-sg"
  }
}

resource "aws_security_group" "backend" {
  name_prefix = "ci-backend-"
  description = "Backend Netdata and restricted SSH"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description     = "Netdata only from frontend"
    from_port       = 19999
    to_port         = 19999
    protocol        = "tcp"
    security_groups = [aws_security_group.frontend.id]
  }
```

```
  ingress {
    description = "SSH from administrator"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_allowed_cidr]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "ci-backend-sg"
  }
}

resource "aws_instance" "frontend" {
  ami                    = data.aws_ssm_parameter.amazon_linux_2023.value
  instance_type          = var.instance_type
  key_name               = var.key_name
  subnet_id              = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.frontend.id]
```

```
  associate_public_ip_address = true

  tags = {
    Name = "c8.local"
    Hostname = "c8.local"
    Role = "frontend"
  }
}

resource "aws_instance" "backend" {
  ami                    = var.ubuntu_2104_ami_id
  instance_type          = var.instance_type
  key_name               = var.key_name
  subnet_id              = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.backend.id]
  associate_public_ip_address = true

  tags = {
    Name = "u21.local"
    Hostname = "u21.local"
    Role = "backend"
  }
}

resource "local_file" "ansible_inventory" {
  filename = "${path.module}/../ansible/inventory.ini"
```

```
  content = <<-EOT
[frontend]
c8.local ansible_host=${aws_instance.frontend.public_ip} ansible_user=ec2-user ansible_ssh_private_key_file=${var.ssh_private_key_path}

[backend]
u21.local ansible_host=${aws_instance.backend.public_ip} ansible_user=${var.ubuntu_2104_username} ansible_ssh_private_key_file=${var.ssh_private_key_path}

[all:vars]
ansible_python_interpreter=auto_silent
EOT
}
```

A few details:

- The frontend receives public HTTP traffic on port 80.
- The backend accepts port 19999 only from the frontend security group.
- Both machines allow SSH only from your specified administrator CIDR.
- The local_file resource generates ansible/inventory.ini from the actual Terraform public IP outputs.
- The Name and Hostname tags identify the instances in AWS; Ansible will also set the actual OS hostnames.

AWS networking prerequisite: your default VPC must have a usable subnet and internet access. If your account has no default VPC, replace this with an explicitly defined VPC, public subnets, routes, and internet gateway.

5.3 Create terraform/outputs.tf

```
output "frontend_public_ip" {
  value = aws_instance.frontend.public_ip
}

output "backend_public_ip" {
  value = aws_instance.backend.public_ip
}

output "backend_private_ip" {
  value = aws_instance.backend.private_ip
}

output "frontend_url" {
  value = "http://${aws_instance.frontend.public_ip}"
}
```

5.4 Create terraform/terraform.tfvars.example

```
aws_region          = "us-east-1"
key_name            = "your-existing-keypair"
ssh_private_key_path = "/secure/path/to/your-key.pem"
ssh_allowed_cidr    = "YOUR_PUBLIC_IP/32"

instance_type       = "t3.micro"

# Supply a verified, existing Ubuntu 21.04 AMI ID.
ubuntu_2104_ami_id  = "ami-REPLACE_WITH_VALID_AMI"
```

ubuntu_2104_username = "ubuntu"

Make a real terraform.tfvars from this example, replace the placeholders, and ensure it is excluded from Git. The SSH key path must be valid on the machine running Terraform and Ansible.

For a Jenkins agent, a credential-bound temporary key file is generally preferable to a permanent key path.

6. Create the Ansible configuration

6.1 Create ansible/ansible.cfg

[defaults]

inventory = ./inventory.ini

host_key_checking = False

timeout = 30

retry_files_enabled = False

[privilege_escalation]

become = True

become_method = sudo

become_ask_pass = False

For this lab, host key checking is disabled to simplify first connections. In production, keep host key verification enabled and manage known hosts securely.

6.2 Create ansible/site.yml

This is the main playbook. It will:

1. Set the hostnames to c8.local and u21.local.
2. Disable SELinux where supported.
3. Stop and disable firewalld if installed.
4. Install and configure Netdata on the backend.

5. Install Nginx and configure reverse proxying on the frontend.

```
---
- name: Configure common operating system settings
  hosts: all
  become: true
  gather_facts: true

  tasks:
    - name: Set hostname to inventory alias
      ansible.builtin.hostname:
        name: "{{ inventory_hostname }}"

    - name: Install SELinux management tools on Amazon Linux
      ansible.builtin.dnf:
        name: python3-libselinux
        state: present
      when: ansible_facts['os_family'] == "RedHat"

    - name: Disable SELinux on supported systems
      ansible.posix.selinux:
        state: disabled
      when: ansible_facts['os_family'] == "RedHat"

    - name: Check whether firewalld is installed
      ansible.builtin.command: rpm -q firewalld
```

```
  register: firewalld_check
  changed_when: false
  failed_when: false
  when: ansible_facts['os_family'] == "RedHat"

- name: Stop and disable firewalld if installed
  ansible.builtin.systemd:
    name: firewalld
    state: stopped
    enabled: false
  when:
    - ansible_facts['os_family'] == "RedHat"
    - firewalld_check.rc == 0

- name: Check whether firewalld is installed on Ubuntu
  ansible.builtin.command: dpkg-query -W -f='${Status}' firewalld
  register: ubuntu_firewalld_check
  changed_when: false
  failed_when: false
  when: ansible_facts['os_family'] == "Debian"

- name: Stop and disable firewalld on Ubuntu if installed
  ansible.builtin.systemd:
    name: firewalld
    state: stopped
    enabled: false
```

```
    when:
      - ansible_facts['os_family'] == "Debian"
      - ubuntu_firewalld_check.rc == 0
      - "'install ok installed' in ubuntu_firewalld_check.stdout"


- name: Install and configure Netdata backend
  hosts: backend
  become: true
  gather_facts: true

  tasks:
    - name: Install prerequisites
      ansible.builtin.package:
        name:
          - curl
          - ca-certificates
        state: present
        update_cache: "{{ ansible_facts['os_family'] == 'Debian' }}"

    - name: Install Netdata using its official installer
      ansible.builtin.shell: |
        set -o pipefail
        curl -fsSL https://get.netdata.cloud/kickstart.sh \
          -o /tmp/netdata-kickstart.sh
        bash /tmp/netdata-kickstart.sh --stable-channel --non-interactive
```

```
  args:
    executable: /bin/bash
    creates: /usr/sbin/netdata

- name: Configure Netdata to listen on port 19999
  ansible.builtin.blockinfile:
    path: /etc/netdata/netdata.conf
    create: true
    marker: "# {mark} ANSIBLE MANAGED NETDATA WEB CONFIG"
    block: |
      [web]
          default port = 19999
          bind to = *
  notify: Restart Netdata

- name: Enable and start Netdata
  ansible.builtin.systemd:
    name: netdata
    state: started
    enabled: true

- name: Apply pending Netdata configuration
  ansible.builtin.meta: flush_handlers

- name: Verify Netdata responds locally
  ansible.builtin.uri:
```

```
      url: http://127.0.0.1:19999/api/v1/info
      status_code: 200
    register: netdata_health
    retries: 12
    delay: 5
    until: netdata_health.status == 200

  handlers:
    - name: Restart Netdata
      ansible.builtin.systemd:
        name: netdata
        state: restarted



- name: Configure Nginx frontend reverse proxy
  hosts: frontend
  become: true
  gather_facts: true

  vars:
    backend_ip: "{{ hostvars[groups['backend'][0]]['ansible_host'] }}"

  tasks:
    - name: Install Nginx
      ansible.builtin.package:
        name: nginx
```

```
    state: present

- name: Configure Nginx reverse proxy
  ansible.builtin.copy:
    dest: /etc/nginx/conf.d/netdata-proxy.conf
    owner: root
    group: root
    mode: "0644"
    content: |
      server {
          listen 80 default_server;
          server_name _;

          location / {
              proxy_pass http://{{ backend_ip }}:19999;
              proxy_http_version 1.1;
              proxy_set_header Host $host;
              proxy_set_header X-Real-IP $remote_addr;
              proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
              proxy_set_header X-Forwarded-Proto $scheme;
          }
      }
  notify: Restart Nginx

- name: Enable and start Nginx
  ansible.builtin.systemd:
```

```
    name: nginx
    state: started
    enabled: true

  - name: Validate Nginx configuration
    ansible.builtin.command: nginx -t
    changed_when: false

  - name: Apply pending Nginx configuration
    ansible.builtin.meta: flush_handlers

handlers:
  - name: Restart Nginx
    ansible.builtin.systemd:
      name: nginx
      state: restarted
```

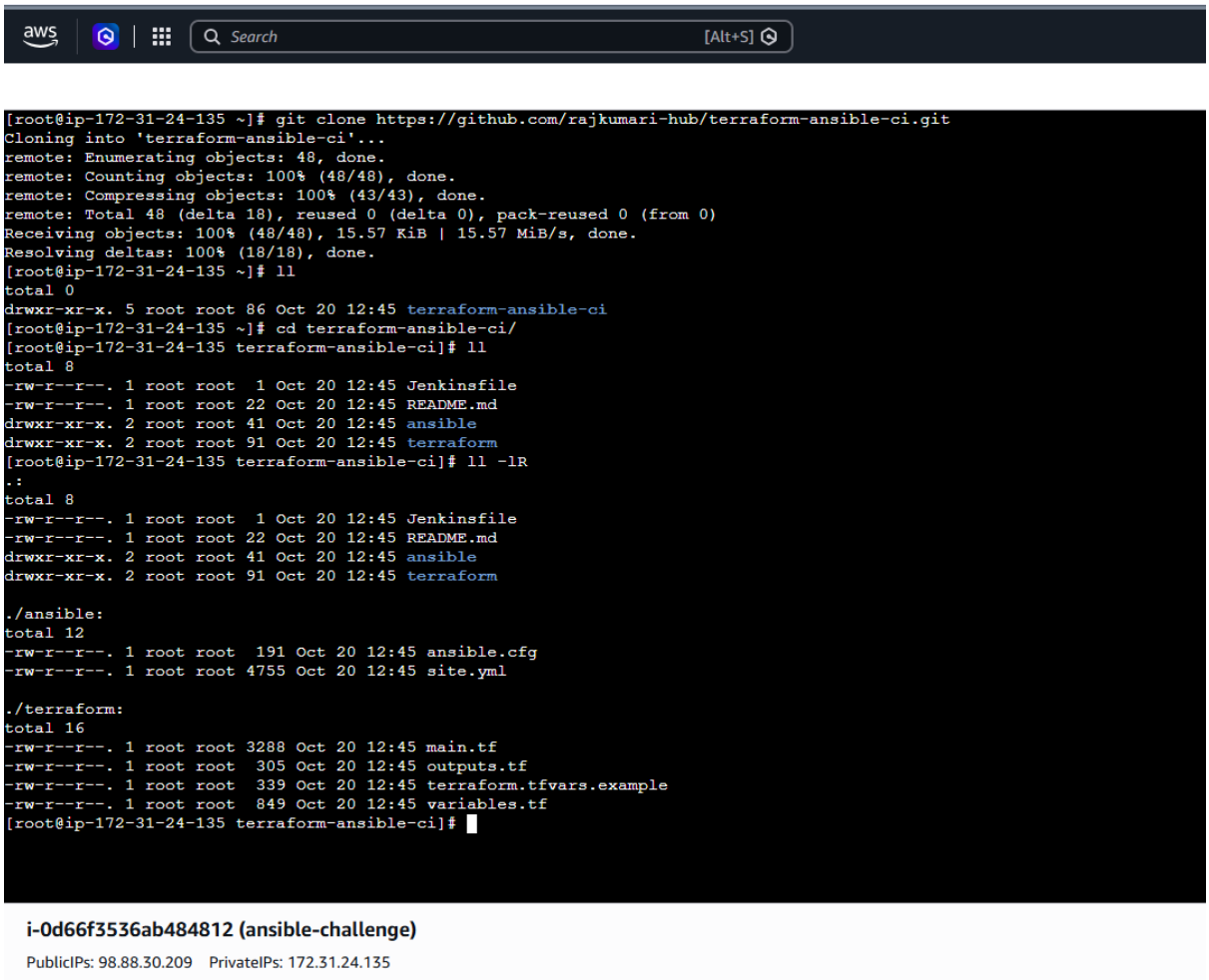
Important implementation notes

- Install the ansible.posix collection on the control node because the SELinux task uses its module:

```
ansible-galaxy collection install ansible.posix
```

- SELinux is relevant to Amazon Linux, not Ubuntu. The SELinux task is therefore conditional.
- Stopping firewalld does not disable Ubuntu's ufw. If UFW is active, it is a separate service that must be handled separately.
- The Netdata installer is downloaded from Netdata's official installation endpoint. Its kickstart script configures the appropriate supported package installation method.

- Netdata's listener is configured to accept connections on port 19999. The AWS security group still restricts access to the frontend. Disabling the OS firewall is not a substitute for network-level security controls.
- On Amazon Linux 2023, disabling SELinux can require a reboot before the change takes full effect. A production playbook should schedule and verify that reboot appropriately.



7. Test Terraform and generate the inventory

Before putting the pipeline into Jenkins, test it manually from the project root.

First, prepare your actual variables file:

cp terraform/terraform.tfvars.example terraform/terraform.tfvars

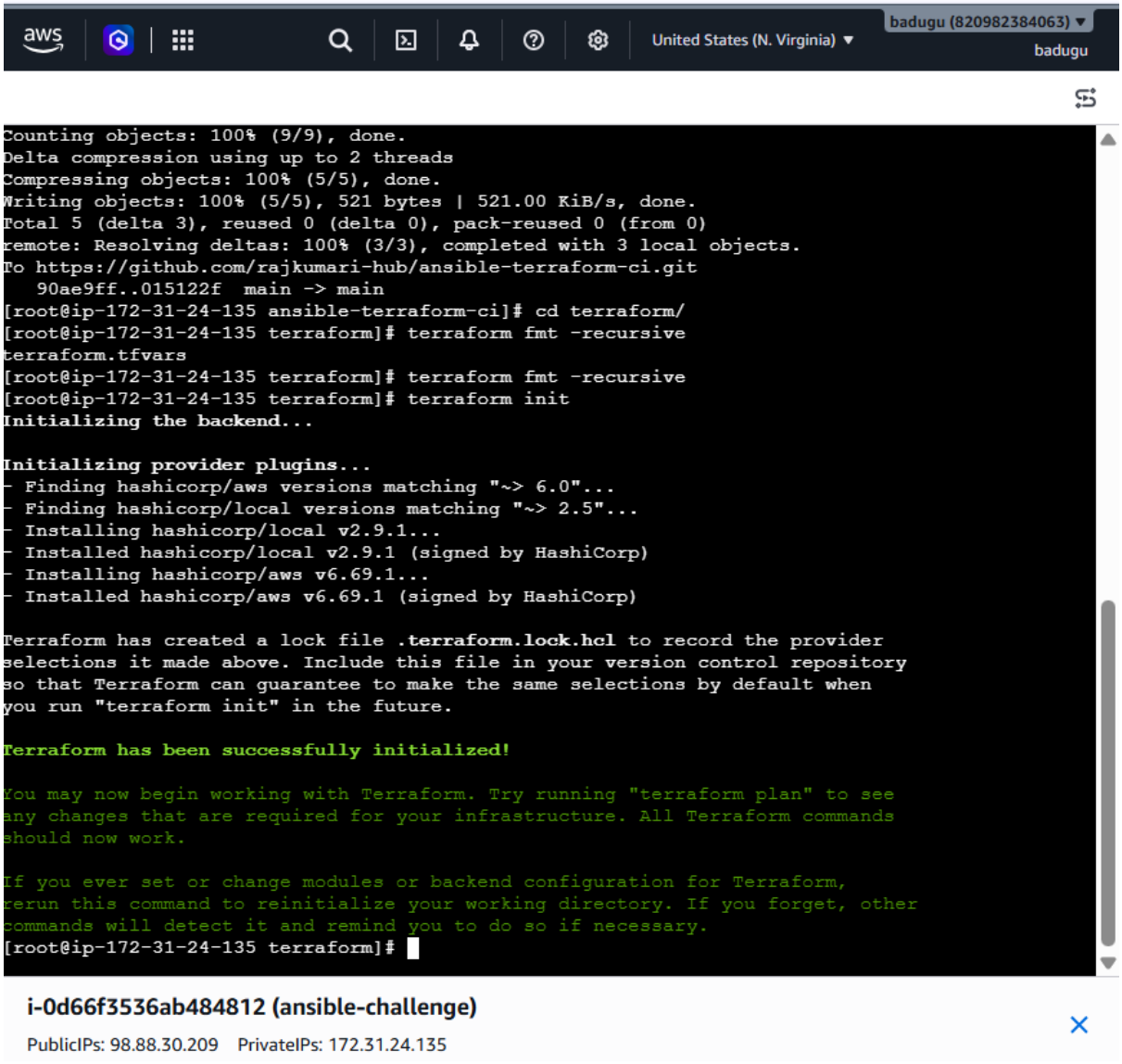
Edit it to set the real SSH key pair, allowed IP range, and valid Ubuntu AMI ID.

Then run:

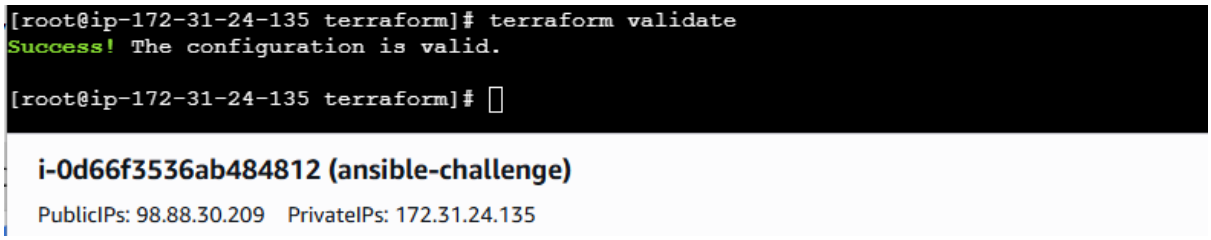
cd terraform

terraform fmt -recursive

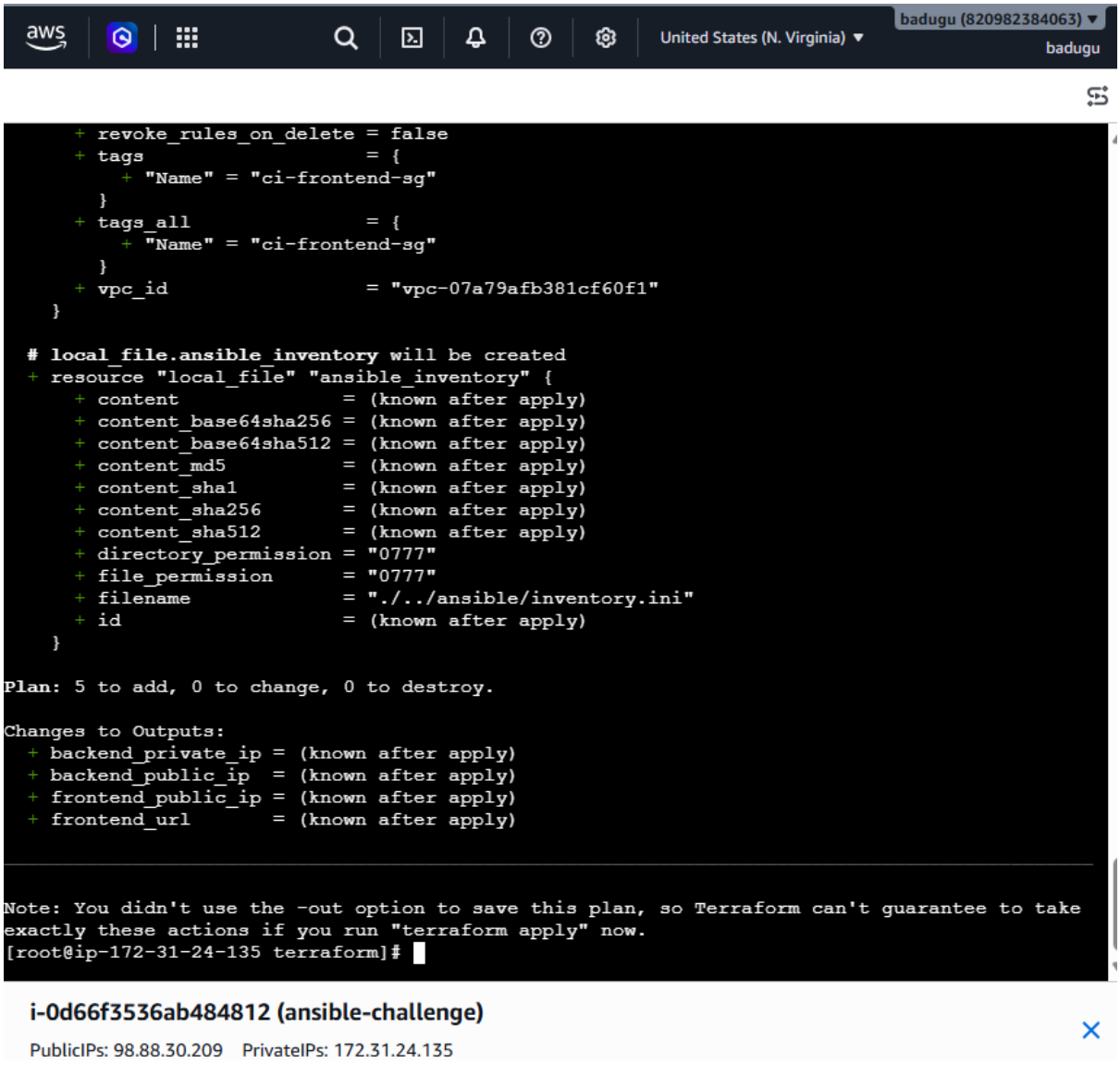
terraform init



terraform validate



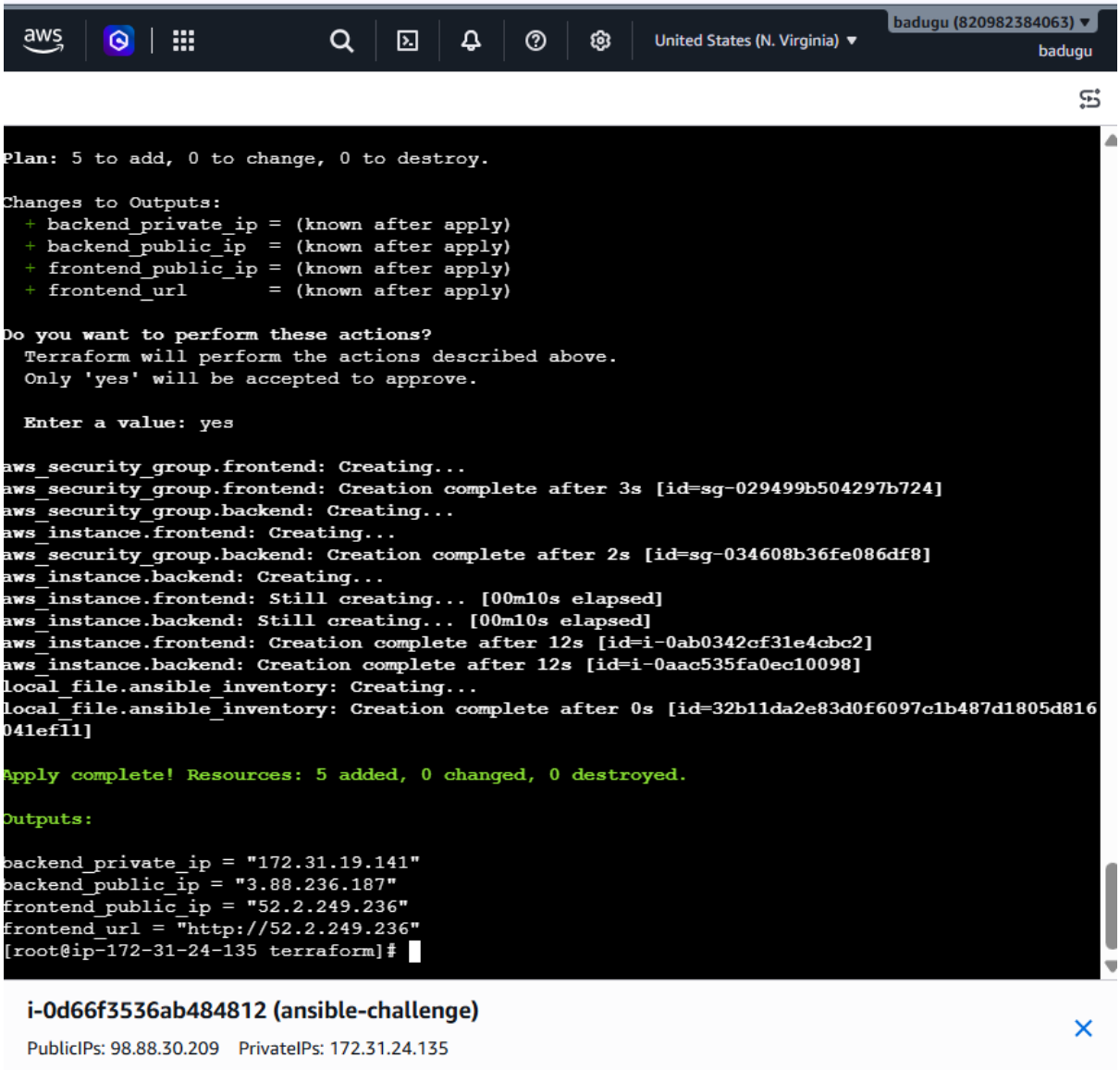
terraform plan



Review the plan carefully. If it shows the two expected instances and security groups, apply it:

terraform apply

Enter yes when prompted.



Inspect the generated inventory:

cat ../ansible/inventory.ini

It should resemble this, with actual public IPs substituted:

[frontend]

c8.local ansible_host=FRONTEND_PUBLIC_IP ansible_user=ec2-user ansible_ssh_private_key_file=/path/to/key.pem

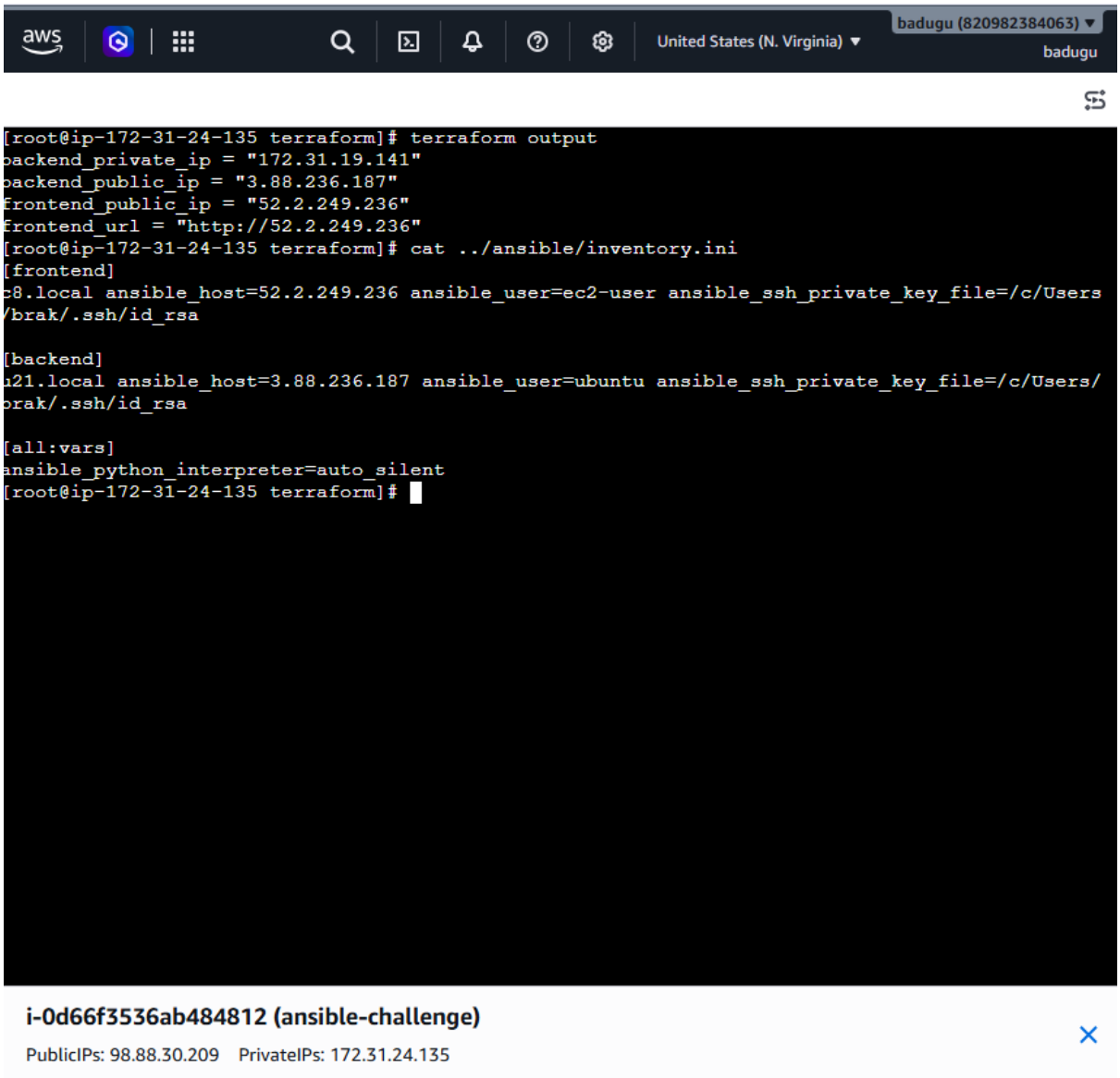
[backend]

u21.local ansible_host=BACKEND_PUBLIC_IP ansible_user=ubuntu ansible_ssh_private_key_file=/path/to/key.pem

[all:vars]

ansible_python_interpreter=auto_silent

Terraform output values are a standard way to pass resource details to other automation tools, and Ansible supports grouping hosts in an inventory file.



7.1 Verify inventory parsing

From the project root:

cd ..

ansible-inventory -i ansible/inventory.ini --graph

Expected group structure:

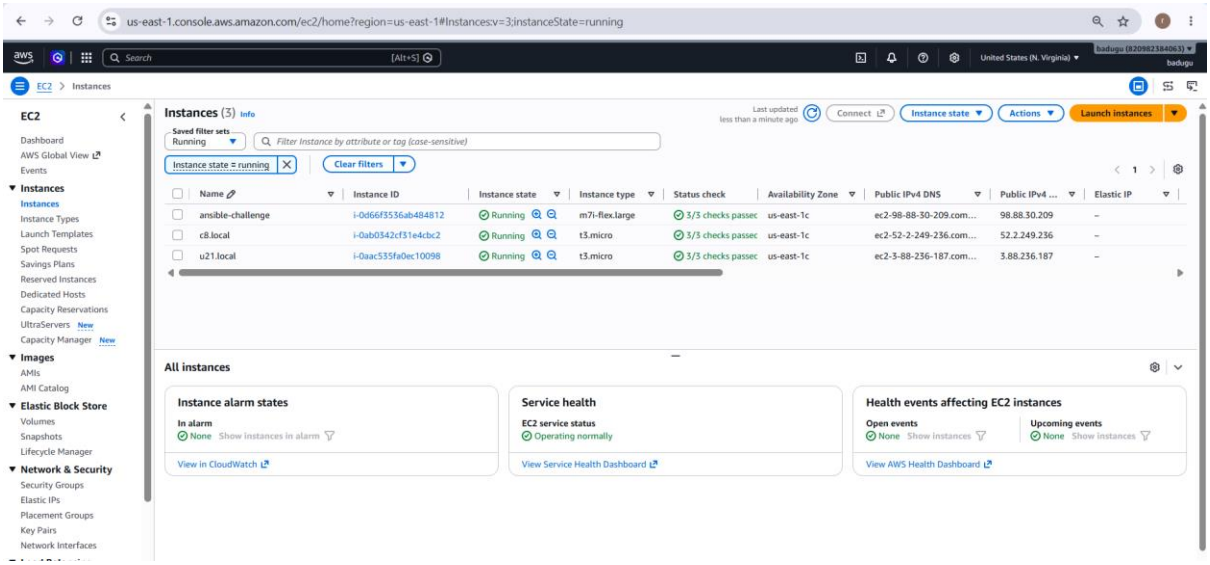
@all:

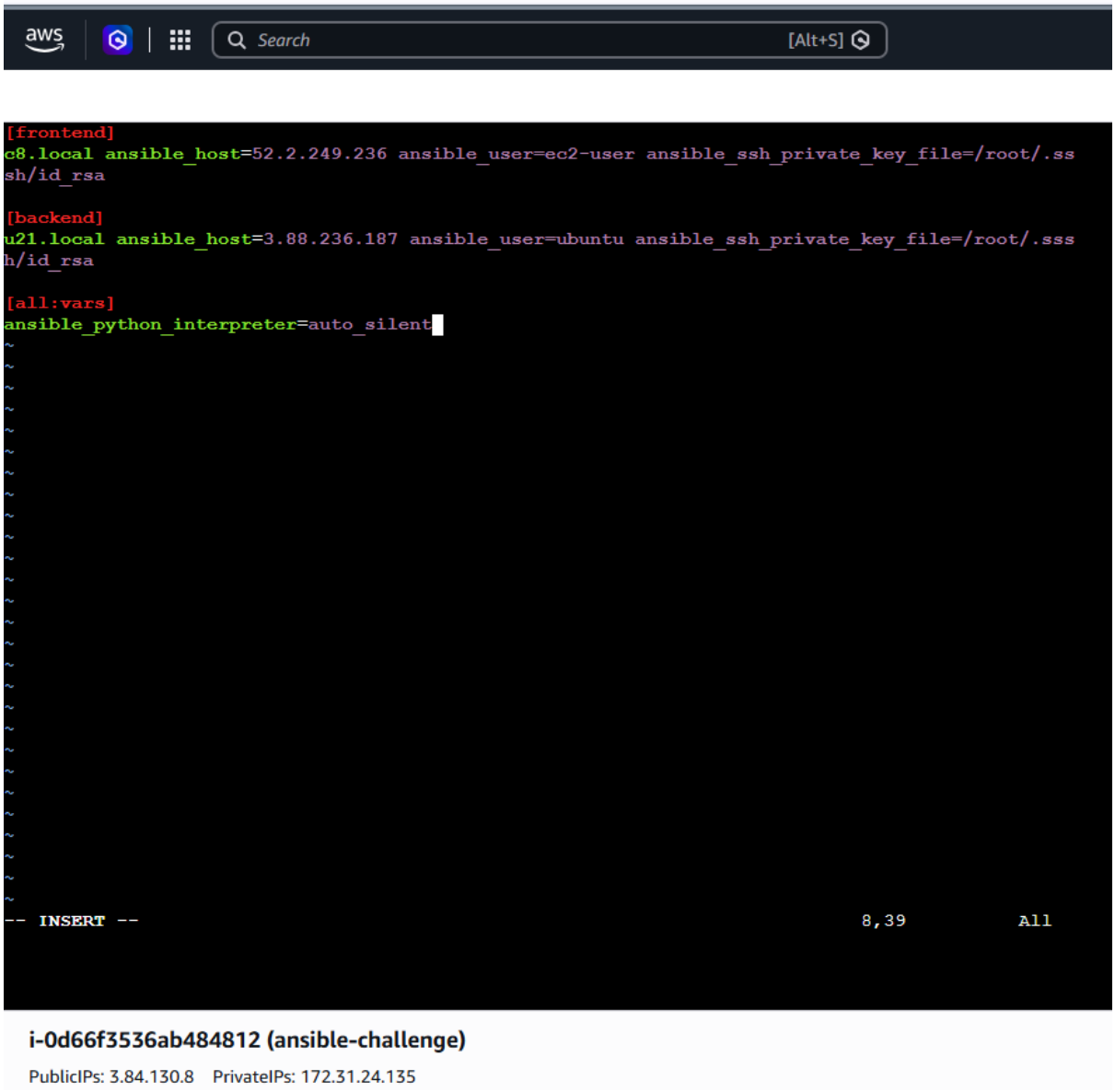
|--@backend:

| |--u21.local

|--@frontend:

| |--c8.local





7.1 Verify inventory parsing

From the project root:

cd ..

ansible-inventory -i ansible/inventory.ini --graph

Expected group structure:

@all:

|--@backend:

| |--u21.local

|--@frontend:

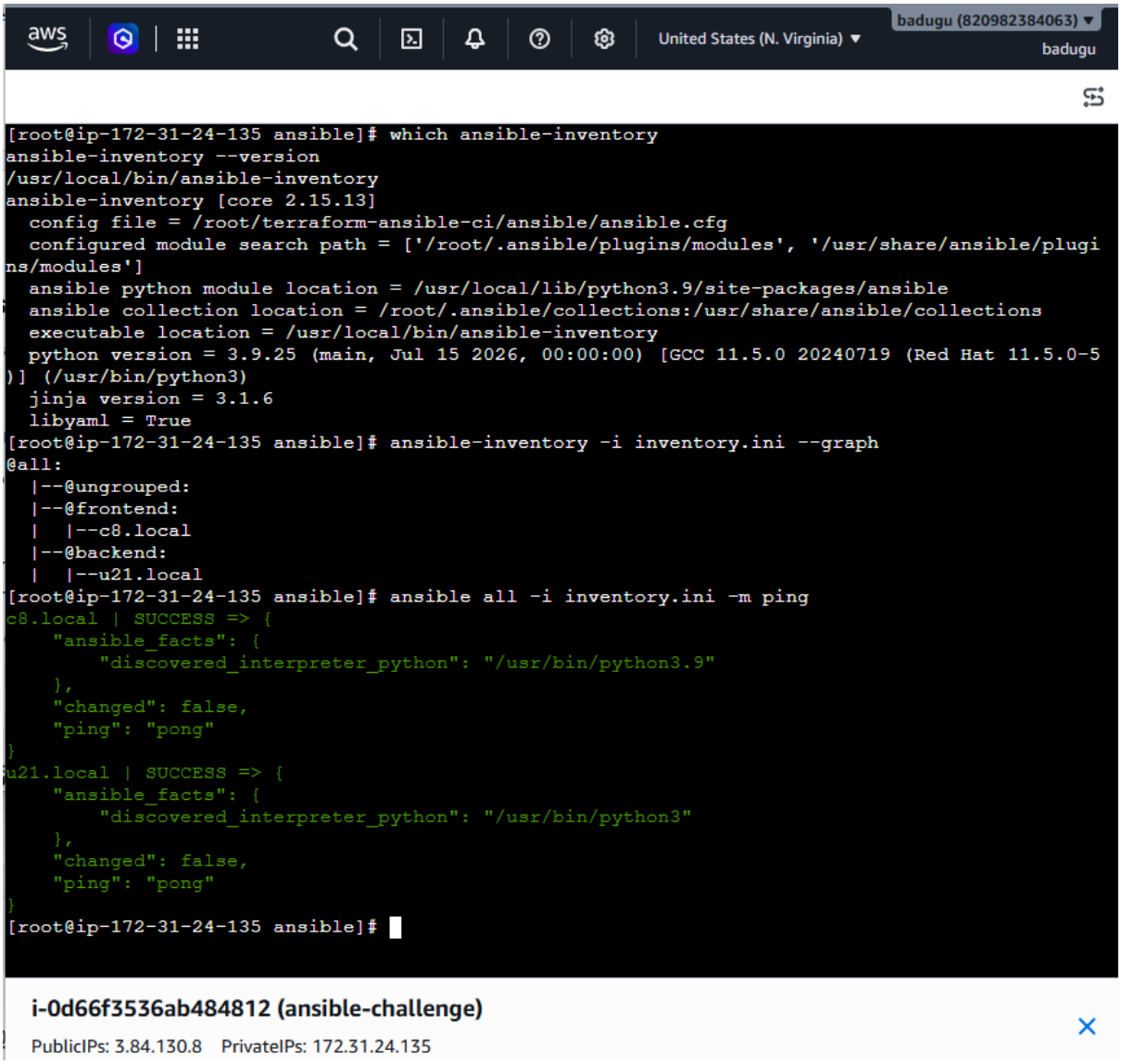
| |--c8.local

7.2 Verify SSH connectivity

ansible all -i ansible/inventory.ini -m ping

If you get UNREACHABLE, verify:

- The instances are running and have reachable public IPs.
- The key pair matches the private key.
- SSH port 22 is allowed from the runner's actual public IP.
- The usernames are correct: ec2-user for Amazon Linux and ubuntu for the Ubuntu image.
- The subnet has a route to an internet gateway and the instance has a public IP.



8. Run Ansible manually

Before creating the CI job, verify the playbook works.

8.1 Install the required collection

ansible-galaxy collection install ansible.posix

8.2 Check playbook syntax

cd ansible

ansible-playbook -i inventory.ini site.yml --syntax-check

8.3 Wait for SSH and run the playbook

Cloud instances can take a few minutes to become ready.

```
ansible all -i inventory.ini -m ansible.builtin.wait_for_connection \
  -a "timeout=300" -e ansible_connection=ssh
```

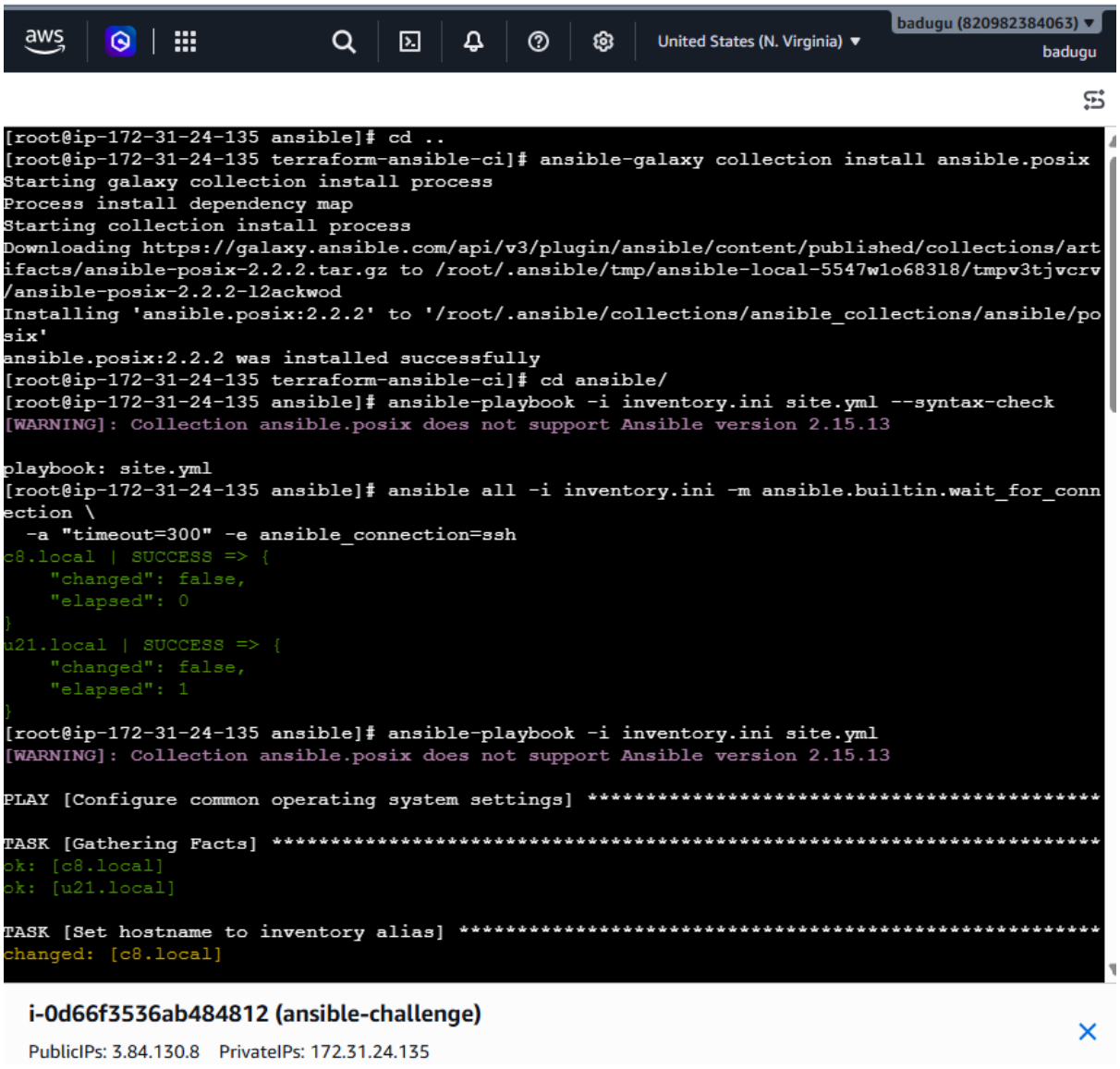
Then execute:

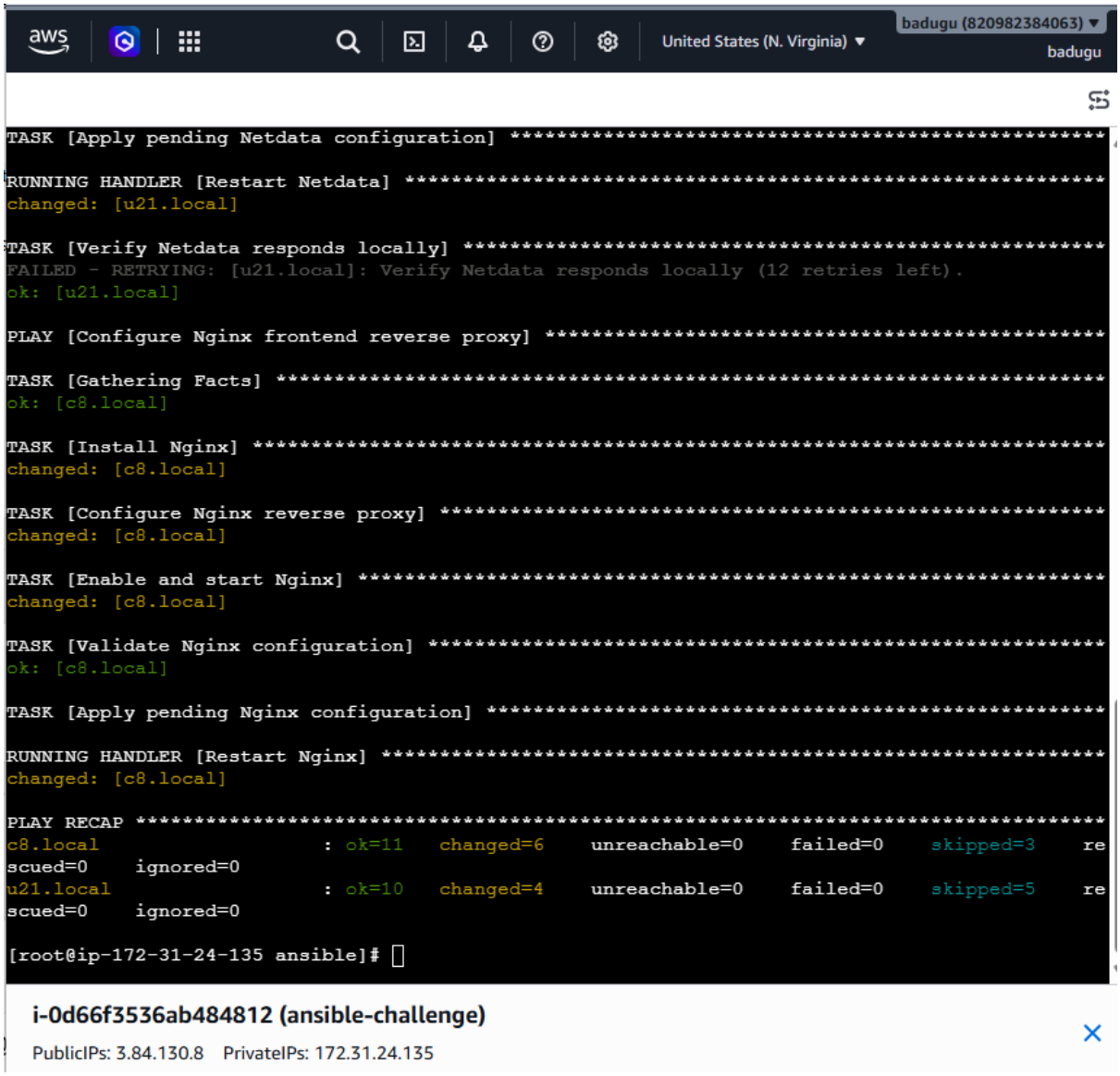
ansible-playbook -i inventory.ini site.yml

If the Netdata task fails because /usr/sbin/netdata is not the executable path used by the installed package, inspect the installation with:

```
ansible backend -i inventory.ini -b -m shell \
  -a "command -v netdata; systemctl status netdata --no-pager"
```

Adjust the task's creates path to match the actual installed binary. Also check that the Ubuntu AMI has working package repositories; Ubuntu 21.04's archived repositories can complicate package installation.


aws
United States (N. Virginia)
badugu (820982384063)
badugu
[root@ip-172-31-24-135 ansible]# cd ..
[root@ip-172-31-24-135 terraform-ansible-ci]# ansible-galaxy collection install ansible.posix
Starting galaxy collection install process
Process install dependency map
Starting collection install process
Downloading https://galaxy.ansible.com/api/v3/plugin/ansible/content/published/collections/artifacts/ansible-posix-2.2.2.tar.gz to /root/.ansible/tmp/ansible-local-5547w1o683l8/tmpv3tjvcrv/ansible-posix-2.2.2-l2ackwod
Installing 'ansible.posix:2.2.2' to '/root/.ansible/collections/ansible_collections/ansible/posix'
ansible.posix:2.2.2 was installed successfully
[root@ip-172-31-24-135 terraform-ansible-ci]# cd ansible/
[root@ip-172-31-24-135 ansible]# ansible-playbook -i inventory.ini site.yml --syntax-check
[WARNING]: Collection ansible.posix does not support Ansible version 2.15.13
playbook: site.yml
[root@ip-172-31-24-135 ansible]# ansible all -i inventory.ini -m ansible.builtin.wait_for_connection \
-a "timeout=300" -e ansible_connection=ssh
c8.local | SUCCESS => {
"changed": false,
"elapsed": 0
}
u21.local | SUCCESS => {
"changed": false,
"elapsed": 1
}
[root@ip-172-31-24-135 ansible]# ansible-playbook -i inventory.ini site.yml
[WARNING]: Collection ansible.posix does not support Ansible version 2.15.13
PLAY [Configure common operating system settings] ****
TASK [Gathering Facts] ****
ok: [c8.local]
ok: [u21.local]
TASK [Set hostname to inventory alias] ****
changed: [c8.local]
i-0d66f3536ab484812 (ansible-challenge)
PublicIPs: 3.84.130.8 PrivateIPs: 172.31.24.135



9. Create the Jenkins CI pipeline

Create a file named Jenkinsfile at the root of the repository.

This pipeline assumes Jenkins has the AWS Credentials plugin and SSH Credentials support installed, with the following credential IDs:

- aws-credentials — AWS access key and secret key, or replace this with an instance role.
- ec2-ssh-key — SSH private key matching the EC2 key pair.

Your Jenkins agent must already have Git, Terraform, Ansible, AWS CLI, curl, and the ansible.posix collection installed.

```
pipeline {
    agent any

    options {
        timestamps()
        disableConcurrentBuilds()
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Validate') {
            steps {
                dir('terraform') {
                    sh '''
                        terraform fmt -check -recursive
                        terraform init
                        terraform validate
                    '''
                }
            }
        }
```

```
stage('Provision EC2 Instances') {
    steps {
        withCredentials([
            sshUserPrivateKey(
                credentialsId: 'ec2-ssh-key',
                keyFileVariable: 'SSH_KEY'
            )
        ]) {
            dir('terraform') {
                sh '''
                    terraform plan \
                      -var-file=terraform.tfvars \
                      -var="ssh_private_key_path=$SSH_KEY" \
                      -out=tfplan

                    terraform apply -auto-approve tfplan
                '''
            }
        }
    }
}

stage('Generate and Verify Inventory') {
    steps {
        withCredentials([
```

```
            sshUserPrivateKey(
                credentialsId: 'ec2-ssh-key',
                keyFileVariable: 'SSH_KEY'
            )
        ]) {
            sh '''
                test -s ansible/inventory.ini
                ansible-inventory \
                  -i ansible/inventory.ini --graph

                ansible all \
                  -i ansible/inventory.ini \
                  -m ansible.builtin.wait_for_connection \
                  -a "timeout=300"
            '''
        }
    }
}

stage('Configure VMs with Ansible') {
    steps {
        withCredentials([
            sshUserPrivateKey(
                credentialsId: 'ec2-ssh-key',
                keyFileVariable: 'SSH_KEY'
            )
```

```
        ]) {
            sh '''
                ansible-galaxy collection install ansible.posix

                ansible-playbook \
                 -i ansible/inventory.ini \
                 ansible/site.yml \
                 --syntax-check

                ansible-playbook \
                 -i ansible/inventory.ini \
                 ansible/site.yml
            '''
        }
    }
}

stage('Verify Deployment') {
    steps {
        sh '''
            FRONTEND_IP=$(cd terraform && terraform output -raw
frontend_public_ip)

            echo "Testing Nginx reverse proxy at $FRONTEND_IP"

            curl --fail --show-error --silent \
```

```
                --retry 12 --retry-delay 5 \
                --retry-connrefused \
                "http://$FRONTEND_IP/api/v1/info" \
                -o netdata-response.json

              cat netdata-response.json
            '''
        }
    }
}

post {
    success {
        echo 'Deployment succeeded: frontend and backend are configured.'
    }

    failure {
        echo 'Pipeline failed. Review Terraform and Ansible logs before retrying.'
    }

    always {
        archiveArtifacts(
            artifacts: 'netdata-response.json',
            allowEmptyArchive: true
        )
    }
```

```
    }
}
```




```
pipeline {
    agent any

    options {
        timestamps()
        disableConcurrentBuilds()
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Validate') {
            steps {
                dir('terraform') {
                    sh '''
                        terraform fmt -check -recursive
                        terraform init
                        terraform validate
                    '''
                }
            }
        }

        stage('Provision EC2 Instances') {
            steps {
                withCredentials([
                    sshUserPrivateKey(
                        credentialsId: 'ec2-ssh-key',
                        keyFileVariable: 'SSH_KEY'
                    )
                ]) {
                    dir('terraform') {
:wq
```



Jenkins configuration notes

1. Create a Pipeline job and connect your Git repository.
2. Set the pipeline definition to Pipeline script from SCM.
3. Select Git, enter the repository URL, and choose the branch containing the Jenkinsfile.
4. Add the AWS and SSH credentials using the exact IDs referenced above.

5. Ensure the AWS credentials are available to Terraform. For static AWS credentials, use the AWS Credentials plugin's environment binding or an IAM role on the Jenkins agent.
6. Keep terraform.tfvars available to the build agent through a secure configuration mechanism. Do not commit private keys or secrets.

The Jenkinsfile is a starting point for a lab pipeline. It assumes the generated inventory and Terraform state remain available on the same agent workspace throughout the build. For a reliable team or production pipeline, configure remote Terraform state with locking, use a dedicated workspace or agent, and ensure the SSH credential is available during the Ansible stage as well.

10. Verify the deployment

Once Jenkins completes successfully, verify the actual machine configuration.



10.1 Check the hostnames

ansible all -i ansible/inventory.ini -b -m command -a "hostname"

Expected:

c8.local

u21.local

10.2 Check Nginx

ansible frontend -i ansible/inventory.ini -b -m command -a "systemctl is-active nginx"

Expected: active

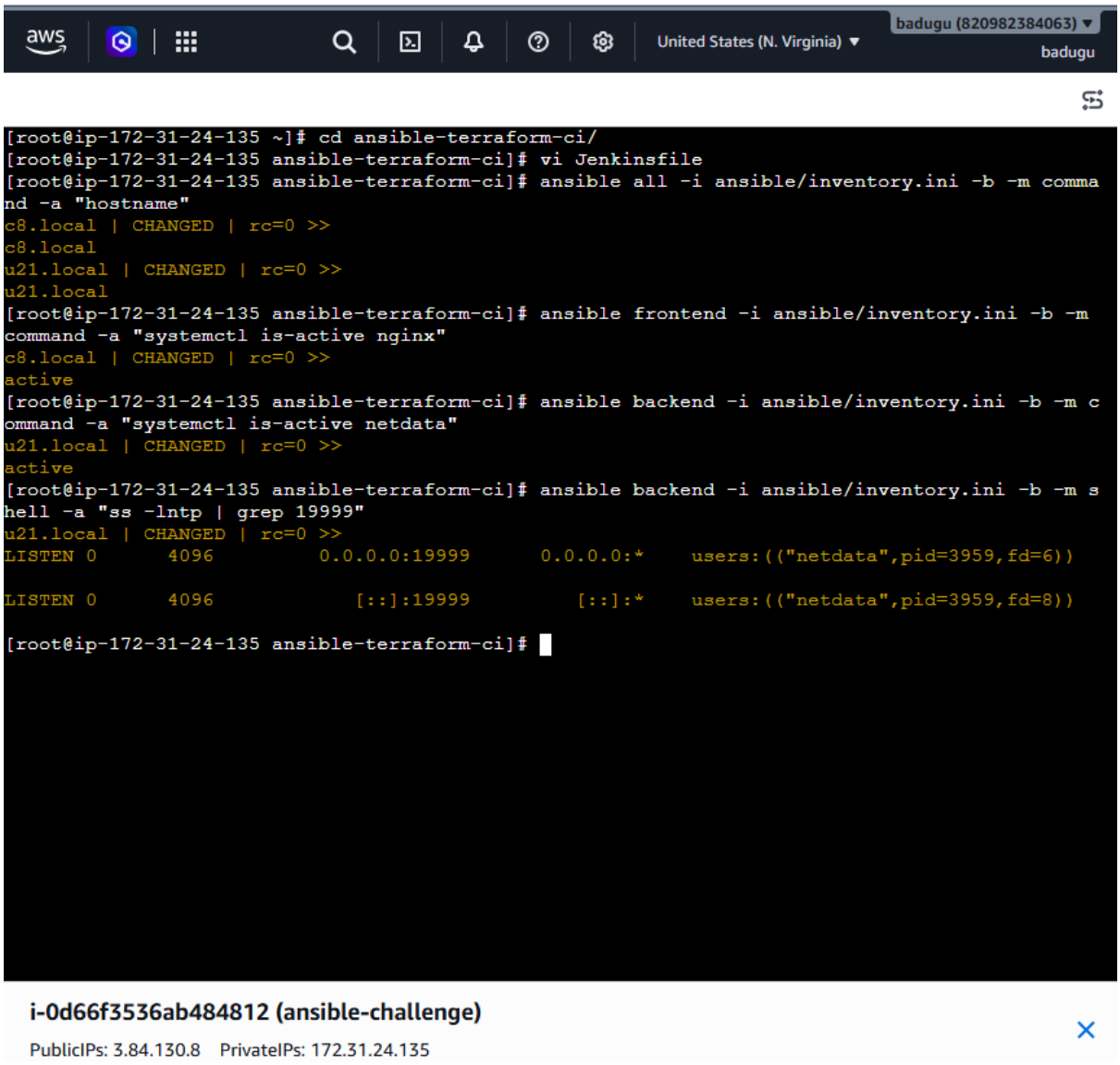
10.3 Check Netdata

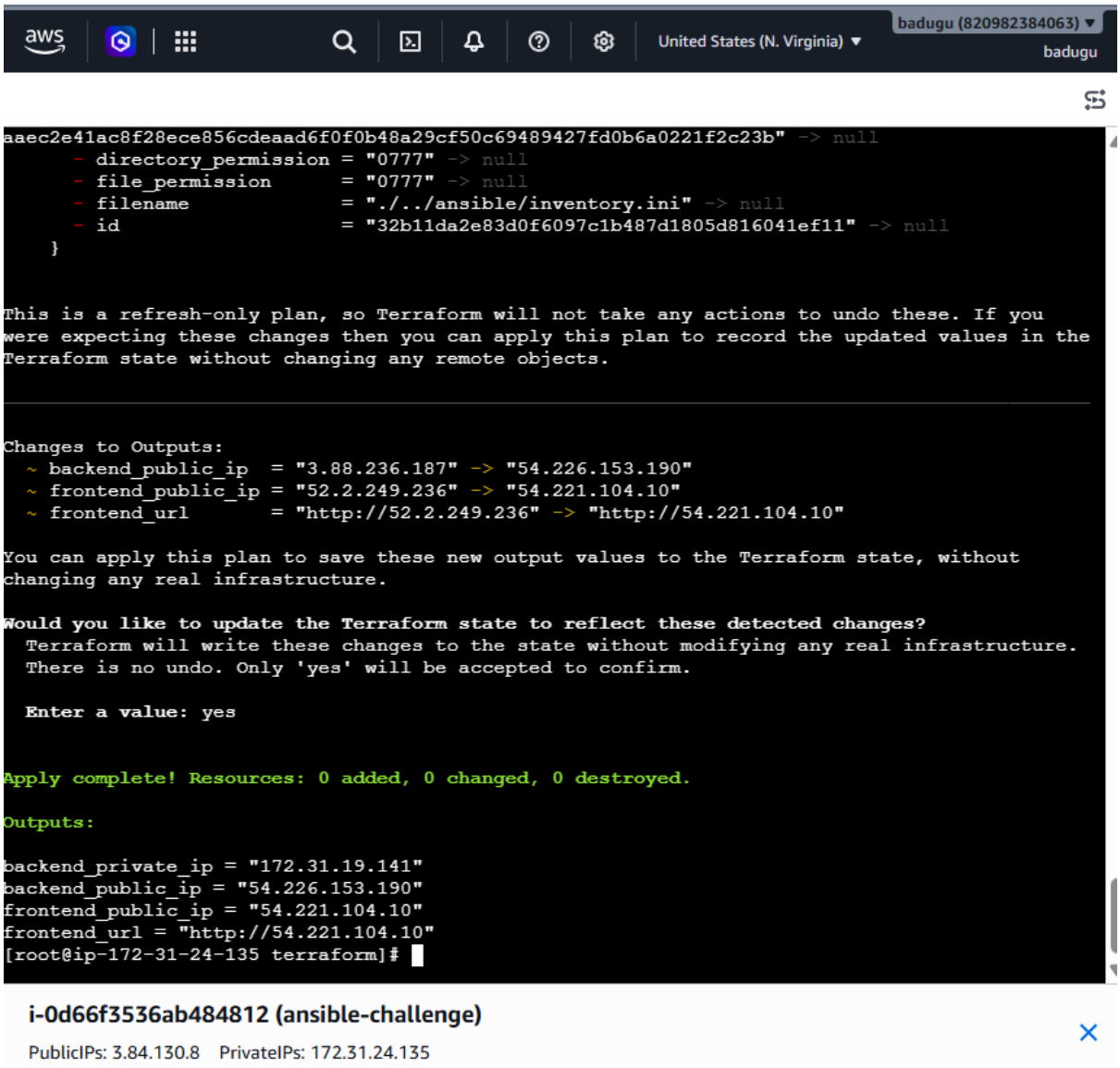
ansible backend -i ansible/inventory.ini -b -m command -a "systemctl is-active netdata"

Expected: active

Check that Netdata listens on port 19999:

ansible backend -i ansible/inventory.ini -b -m shell -a "ss -lntp | grep 19999"





10.4 Check the reverse proxy

Retrieve the frontend IP:

cd terraform

terraform output -raw frontend_public_ip

Open this URL in your browser:

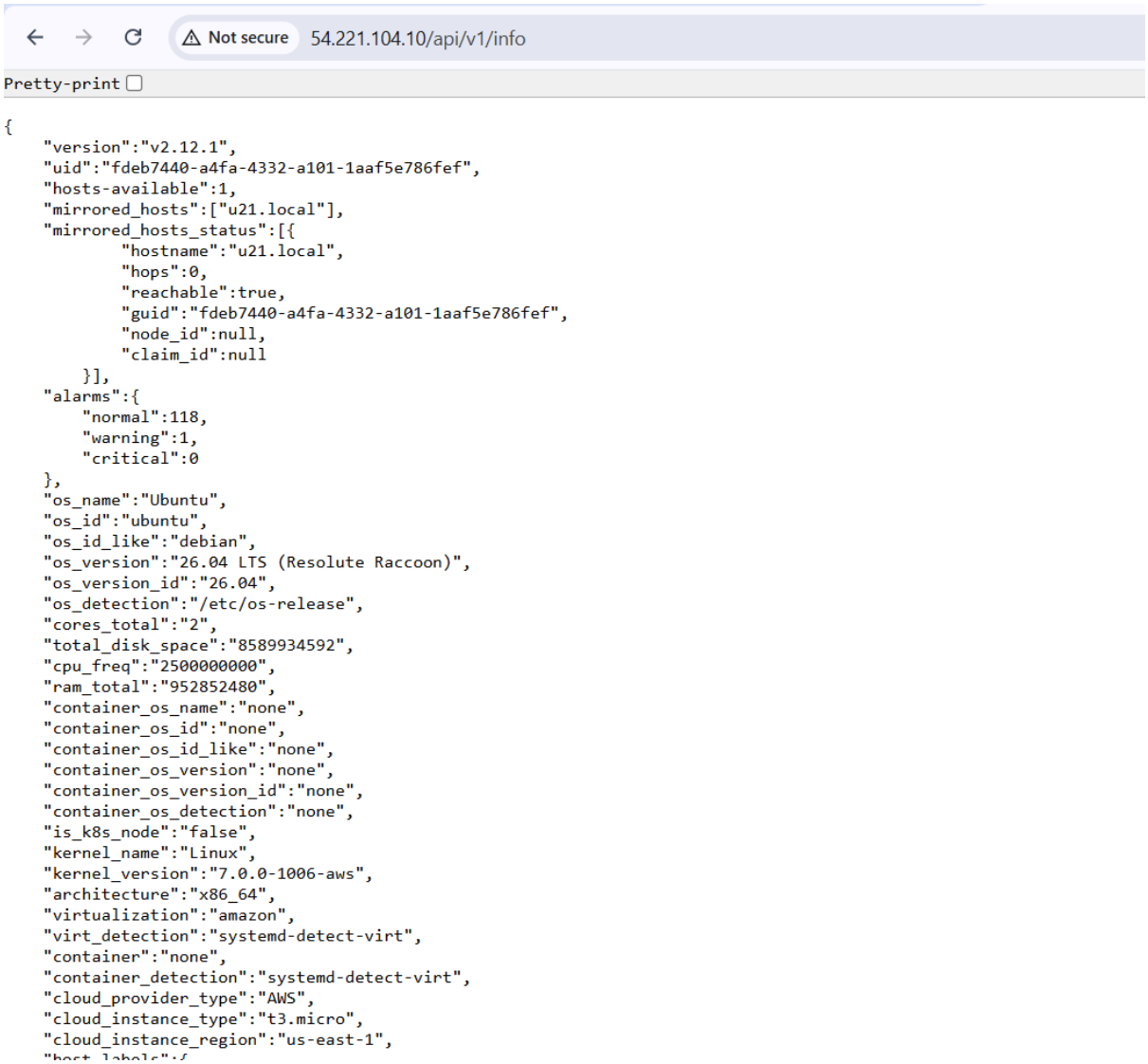
http://FRONTEND_PUBLIC_IP

You should see the Netdata dashboard served through Nginx. You can also test the API endpoint:

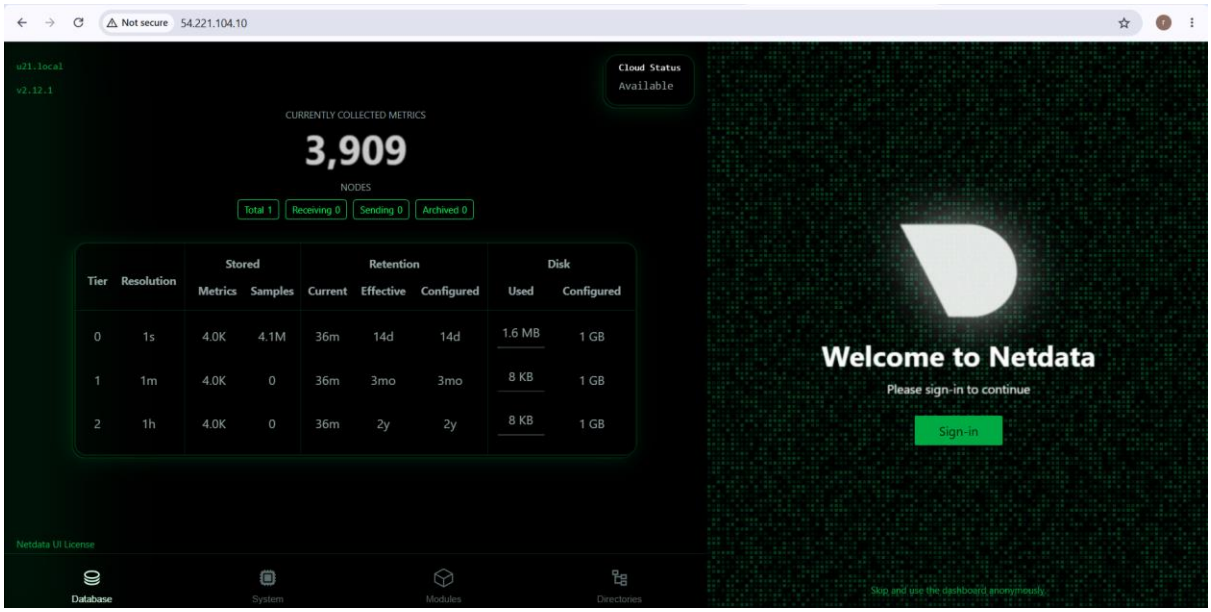
curl -i http://FRONTEND_PUBLIC_IP/api/v1/info

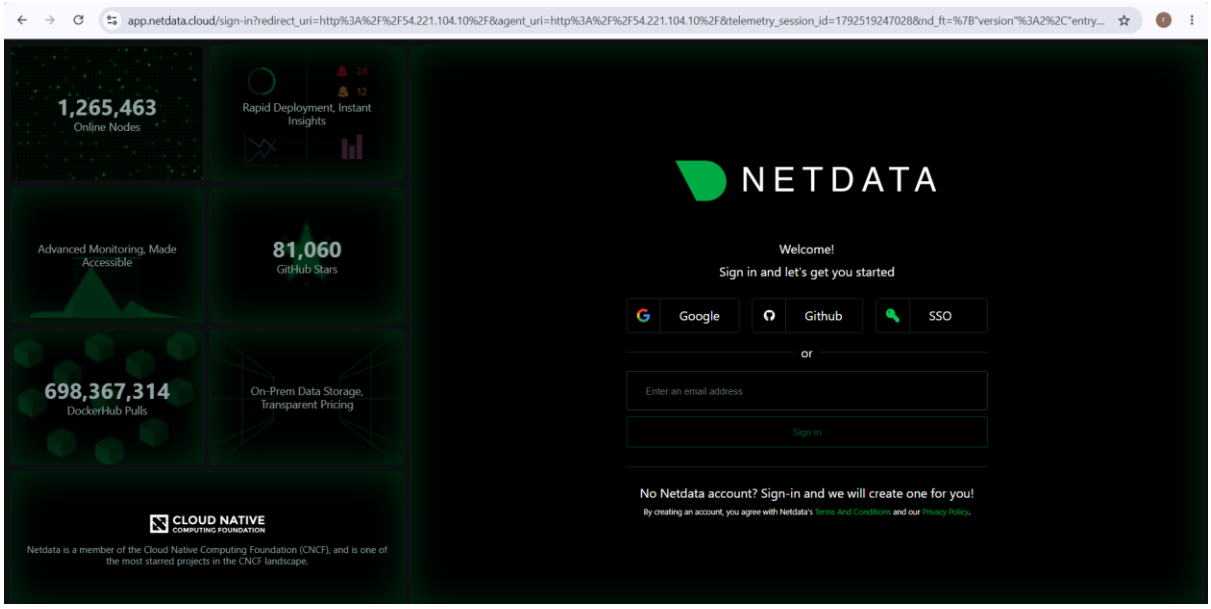
A successful request should return HTTP 200 and Netdata information in JSON format.

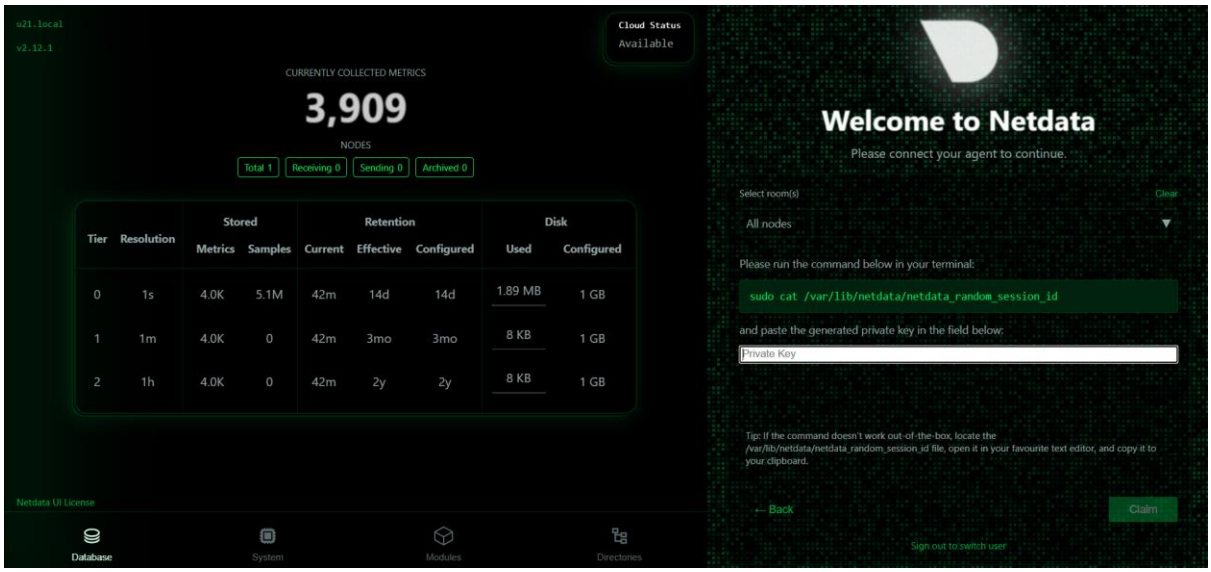
Do not open port 19999 to the entire internet just to make the dashboard work. The browser should access Nginx on port 80, while Nginx accesses Netdata on the backend's private address.




```
{
    "version":"v2.12.1",
    "uid":"fdeb7440-a4fa-4332-a101-1aaf5e786fef",
    "hosts-available":1,
    "mirrored_hosts":["u21.local"],
    "mirrored_hosts_status":[{
            "hostname":"u21.local",
            "hops":0,
            "reachable":true,
            "guid":"fdeb7440-a4fa-4332-a101-1aaf5e786fef",
            "node_id":null,
            "claim_id":null
        }],
    "alarms":{
        "normal":118,
        "warning":1,
        "critical":0
    },
    "os_name":"Ubuntu",
    "os_id":"ubuntu",
    "os_id_like":"debian",
    "os_version":"26.04 LTS (Resolute Raccoon)",
    "os_version_id":"26.04",
    "os_detection":"/etc/os-release",
    "cores_total":"2",
    "total_disk_space":"8589934592",
    "cpu_freq":"2500000000",
    "ram_total":"952852480",
    "container_os_name":"none",
    "container_os_id":"none",
    "container_os_id_like":"none",
    "container_os_version":"none",
    "container_os_version_id":"none",
    "container_os_detection":"none",
    "is_k8s_node":"false",
    "kernel_name":"Linux",
    "kernel_version":"7.0.0-1006-aws",
    "architecture":"x86_64",
    "virtualization":"amazon",
    "virt_detection":"systemd-detect-virt",
    "container":"none",
    "container_detection":"systemd-detect-virt",
    "cloud_provider_type":"AWS",
    "cloud_instance_type":"t3.micro",
    "cloud_instance_region":"us-east-1",
    "host_labels":{
```


Not secure 54.221.104.10
u21.local
v2.12.1
Cloud Status
Available
CURRENTLY COLLECTED METRICS
3,909
NODES
Total 1
Receiving 0
Sending 0
Archived 0
Tier Resolution Stored Metrics Stored Samples Retention Current Retention Effective Retention Configured Disk Used Disk Configured
0 1s 4.0K 4.1M 36m 14d 14d 1.6 MB 1 GB
1 1m 4.0K 0 36m 3mo 3mo 8 KB 1 GB
2 1h 4.0K 0 36m 2y 2y 8 KB 1 GB
Netdata UI License
Database
System
Modules
Directories
Welcome to Netdata
Please sign-in to continue
Sign-in
Skip and use the dashboard anonymously


app.netdata.cloud/sign-in?redirect_uri=http%3A%2F%2F54.221.104.10%2F&agent_uri=http%3A%2F%2F54.221.104.10%2F&telemetry_session_id=1792519247028&nd_ft=%7B"version"%3A2%2C"entry...
1,265,463
Online Nodes
Rapid Deployment, Instant Insights
Advanced Monitoring, Made Accessible
81,060
GitHub Stars
698,367,314
DockerHub Pulls
On-Prem Data Storage, Transparent Pricing
CLOUD NATIVE COMPUTING FOUNDATION
Netdata is a member of the Cloud Native Computing Foundation (CNCF), and is one of the most starred projects in the CNCF landscape.
NETDATA
Welcome!
Sign in and let's get you started
Google
Github
SSO
or
Enter an email address
Sign in
No Netdata account? Sign-in and we will create one for you!
By creating an account, you agree with Netdata's Terms And Conditions and our Privacy Policy.


u21.local
v2.12.1
Cloud Status
Available
CURRENTLY COLLECTED METRICS
3,909
NODES
Total 1
Receiving 0
Sending 0
Archived 0
Tier Resolution Stored Metrics Stored Samples Retention Current Retention Effective Retention Configured Disk Used Disk Configured
0 1s 4.0K 5.1M 42m 14d 14d 1.89 MB 1 GB
1 1m 4.0K 0 42m 3mo 3mo 8 KB 1 GB
2 1h 4.0K 0 42m 2y 2y 8 KB 1 GB
Netdata UI License
Database
System
Modules
Directories
Welcome to Netdata
Please connect your agent to continue.
Select room(s)
Clear
All nodes
Please run the command below in your terminal:
sudo cat /var/lib/netdata/netdata_random_session_id
and paste the generated private key in the field below:
Private Key
Tip: If the command doesn't work out-of-the-box, locate the /var/lib/netdata/netdata_random_session_id file, open it in your favourite text editor, and copy it to your clipboard.
← Back
Claim
Sign out to switch user

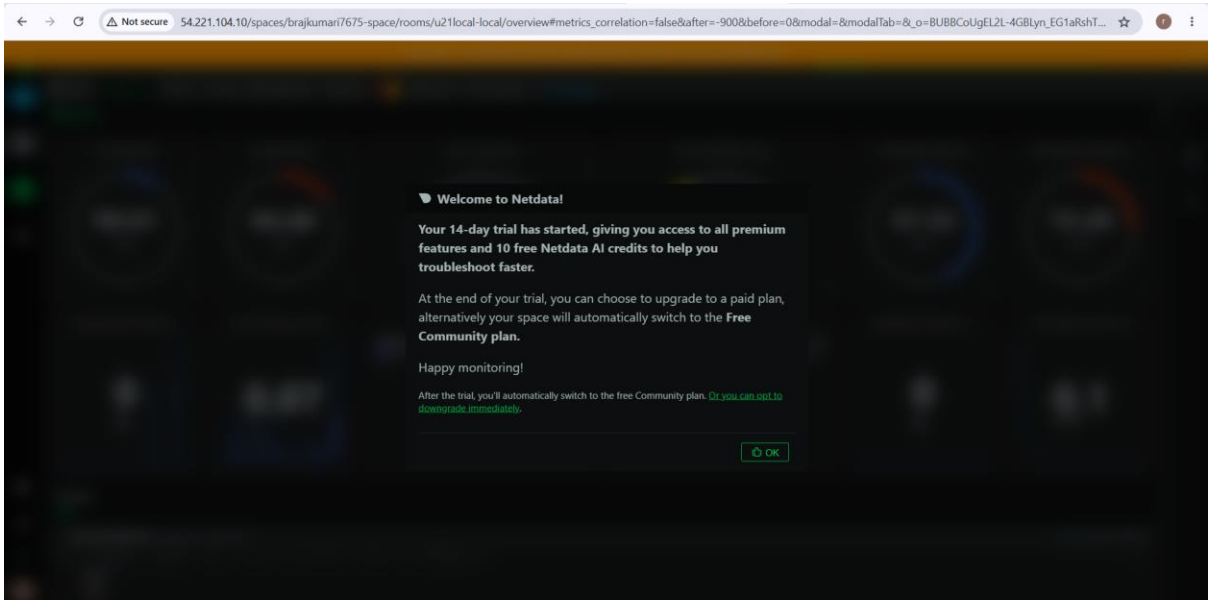


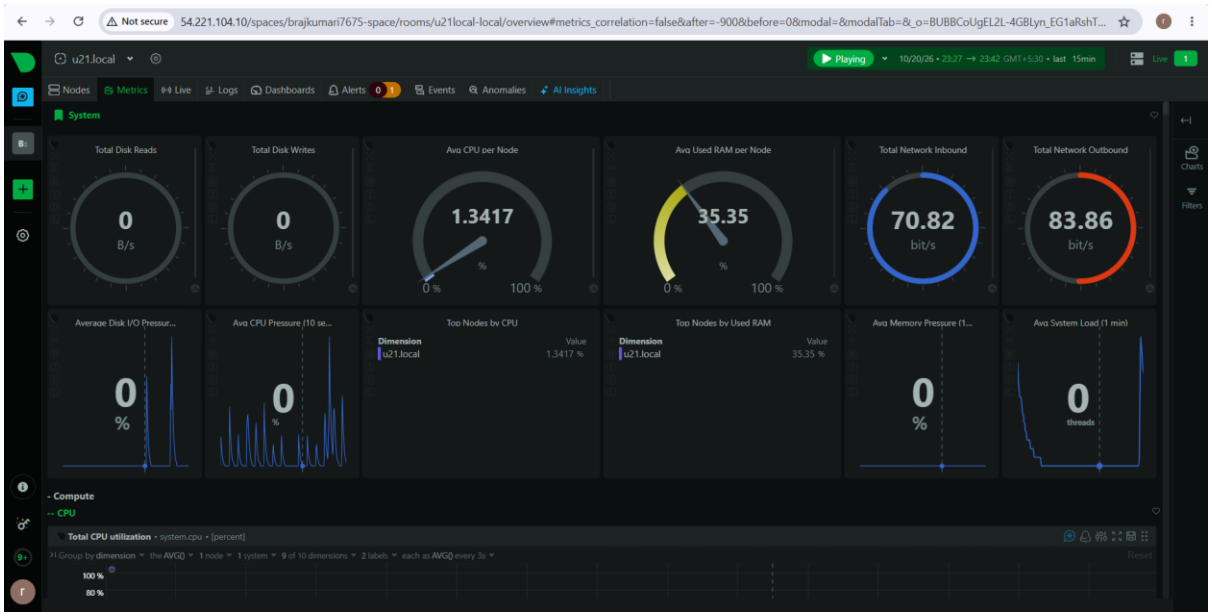

```
           ├─4371 /usr/libexec/netdata/plugins.d/scripts.d.plugin 1
           ├─4372 /usr/libexec/netdata/plugins.d/systemd-units.plugin 1
           ├─4374 /usr/libexec/netdata/plugins.d/otel-plugin 1
           ├─4376 /usr/libexec/netdata/plugins.d/ebpf.plugin 1
           ├─4377 /usr/libexec/netdata/plugins.d/nfacct.plugin 1
           ├─4378 /usr/libexec/netdata/plugins.d/systemd-journal.plugin 1
           ├─4379 /usr/libexec/netdata/plugins.d/debugfs.plugin 1
           ├─4384 "spawn-setns                                  " " "
           ├─4422 /usr/libexec/netdata/plugins.d/otel-plugin worker ledger --socket /run/ne…
           └─4440 /usr/libexec/netdata/plugins.d/otel-plugin worker ingestor --socket /run/…

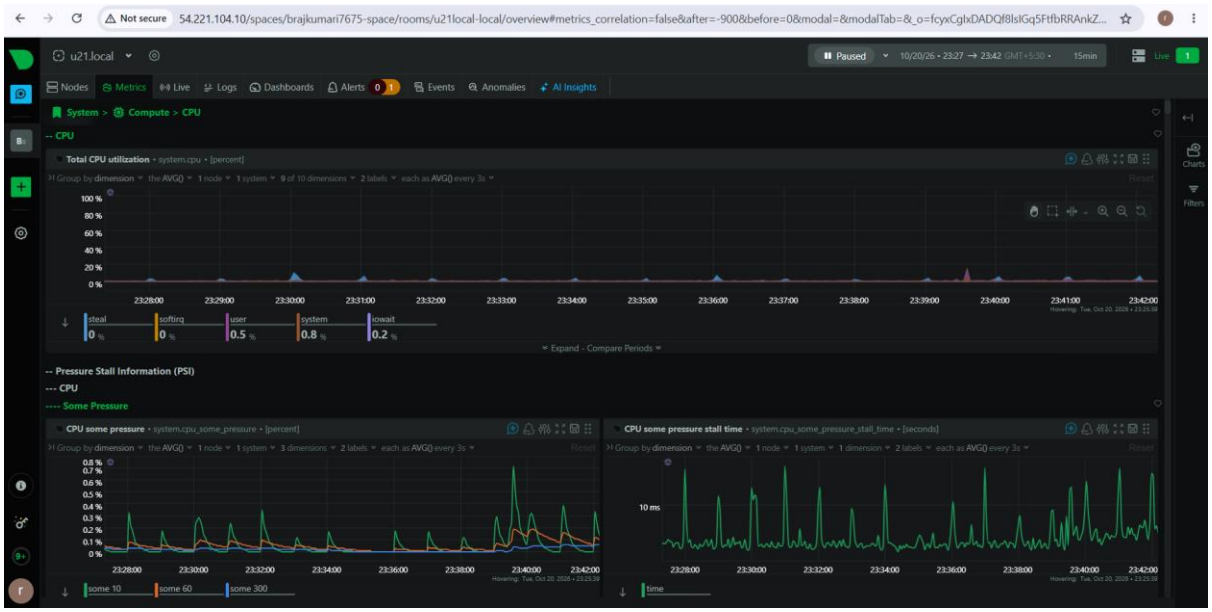
Oct 20 18:01:45 u21.local netdata[4371]: level=error msg="start watching '/etc/netdata/s…=file
Oct 20 18:02:45 u21.local netdata[4371]: level=error msg="start watching '/etc/netdata/s…=file
Oct 20 18:03:45 u21.local netdata[4371]: level=error msg="start watching '/etc/netdata/s…=file
Oct 20 18:04:45 u21.local netdata[4371]: level=error msg="start watching '/etc/netdata/s…=file
Oct 20 18:05:45 u21.local netdata[4371]: level=error msg="start watching '/etc/netdata/s…=file
Oct 20 18:06:45 u21.local netdata[4371]: level=error msg="start watching '/etc/netdata/s…=file
Oct 20 18:07:45 u21.local netdata[4371]: level=error msg="start watching '/etc/netdata/s…=file
Oct 20 18:08:45 u21.local netdata[4371]: level=error msg="start watching '/etc/netdata/s…=file
Oct 20 18:09:33 u21.local netdata[13168]: time=2026-10-20T18:09:33.498Z comm=cgroup-name leve…
Oct 20 18:09:34 u21.local netdata[13258]: time=2026-10-20T18:09:34.480Z comm=cgroup-name leve…
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@u21:~$ sudo ls -la /etc/netdata/
total 64
drwxr-xr-x  10 root root 4096 Oct 20 17:25 .
drwxr-xr-x 109 root root 4096 Oct 20 17:25 ..
-rw-r--r--   1 root root   80 Oct  8 15:07 .install-type
drwxr-xr-x   2 root root 4096 Oct  8 15:50 charts.d
drwxr-xr-x   2 root root 4096 Oct  8 15:33 custom-plugins.d
-rwxr-xr-x   1 root root 9074 Oct  8 15:06 edit-config
drwxr-xr-x   2 root root 4096 Oct  8 15:33 go.d
drwxr-xr-x   2 root root 4096 Oct  8 15:33 health.d
-rw-r--r--   1 root root 2021 Oct  8 15:06 netdata-updater.conf
-rw-r--r--   1 root root  457 Oct 20 17:25 netdata.conf
drwxr-xr-x   3 root root 4096 Oct 20 17:25 otel.d
drwxr-xr-x   2 root root 4096 Oct  8 15:50 python.d
drwxr-xr-x   2 root root 4096 Oct  8 15:33 ssl
drwxr-xr-x   2 root root 4096 Oct  8 15:33 statsd.d
ubuntu@u21:~$ sudo cat /var/lib/netdata/netdata_random_session_id
beb77bf0-b70c-46dc-a235-b58e20e95cf7
ubuntu@u21:~$
```

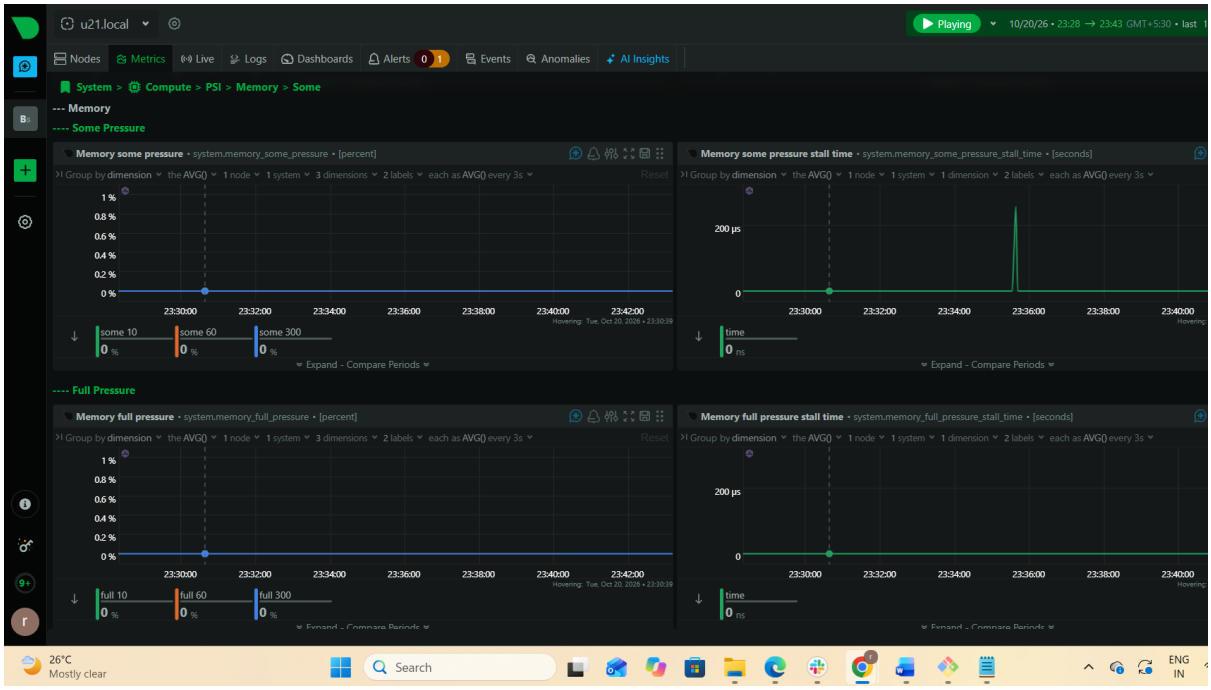
i-0d66f3536ab484812 (ansible-challenge)

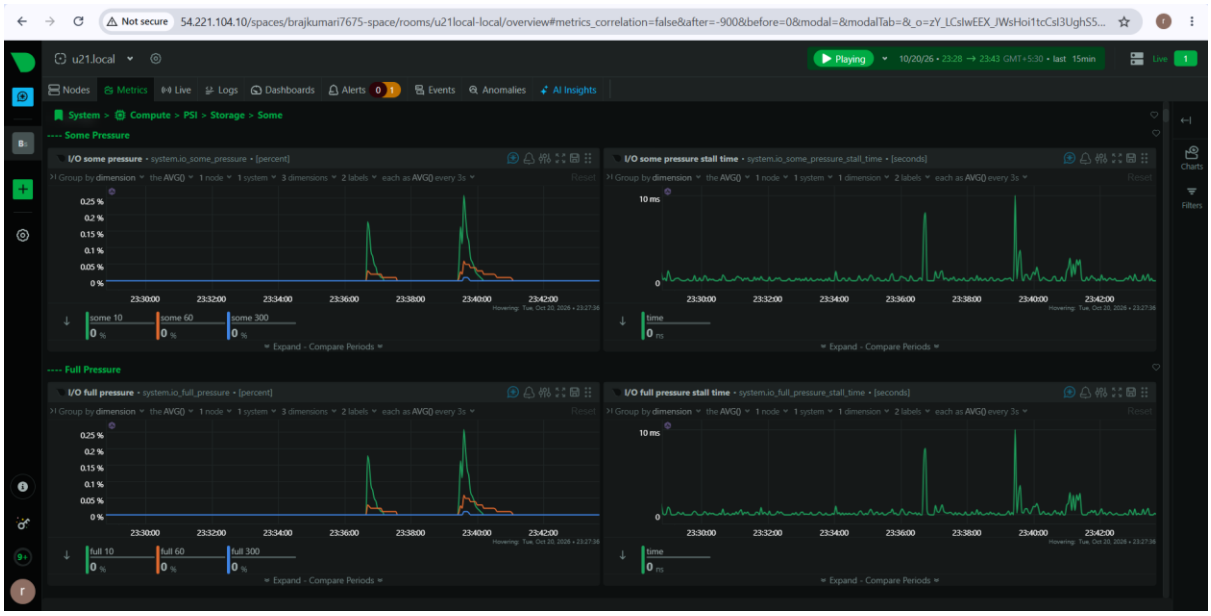
PublicIPs: 3.84.130.8 PrivateIPs: 172.31.24.135



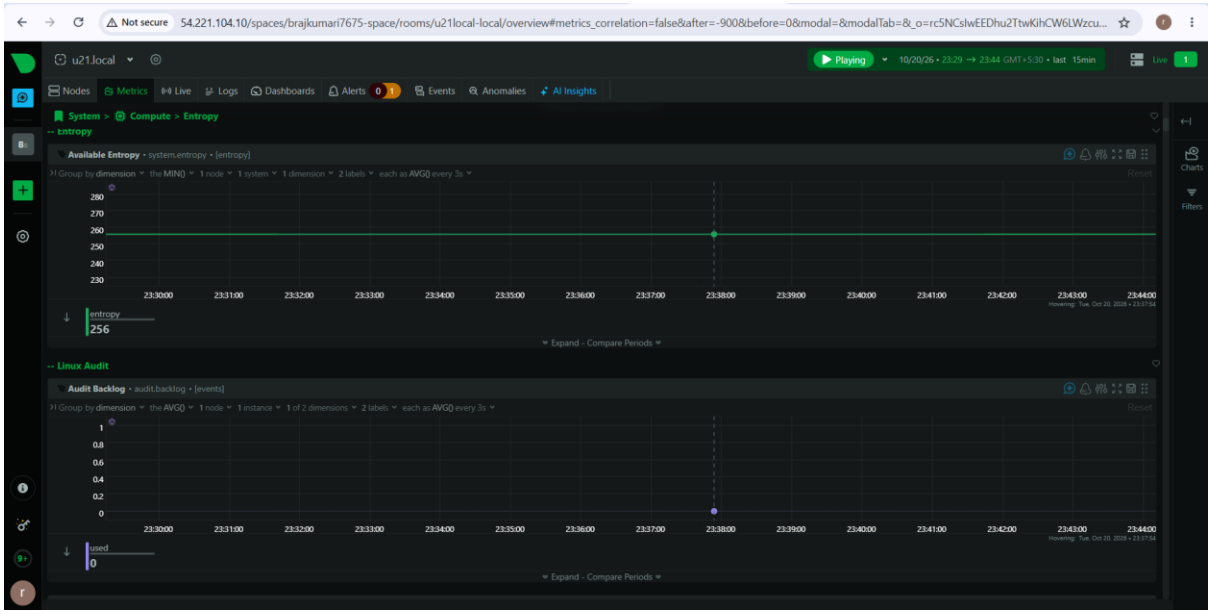

Not secure
54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/overview#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=BUBBCoUgEL2L-4GBLyn_EG1aRshT...
u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
System
Total Disk Reads
0
B/s
Total Disk Writes
0
B/s
Avg CPU per Node
1.3417
%
0 %
100 %
Avg Used RAM per Node
35.35
%
0 %
100 %
Total Network Inbound
70.82
bit/s
Total Network Outbound
83.86
bit/s
Average Disk I/O Pressur...
0
%
Avg CPU Pressure (10 se...
0
%
Top Nodes by CPU
Dimension
Value
u21.local
1.3417 %
Top Nodes by Used RAM
Dimension
Value
u21.local
35.35 %
Avg Memory Pressure (1...
0
%
Avg System Load (1 min)
0
threads
- Compute
-- CPU
Total CPU utilization • system.cpu • [percent]
100 %
80 %

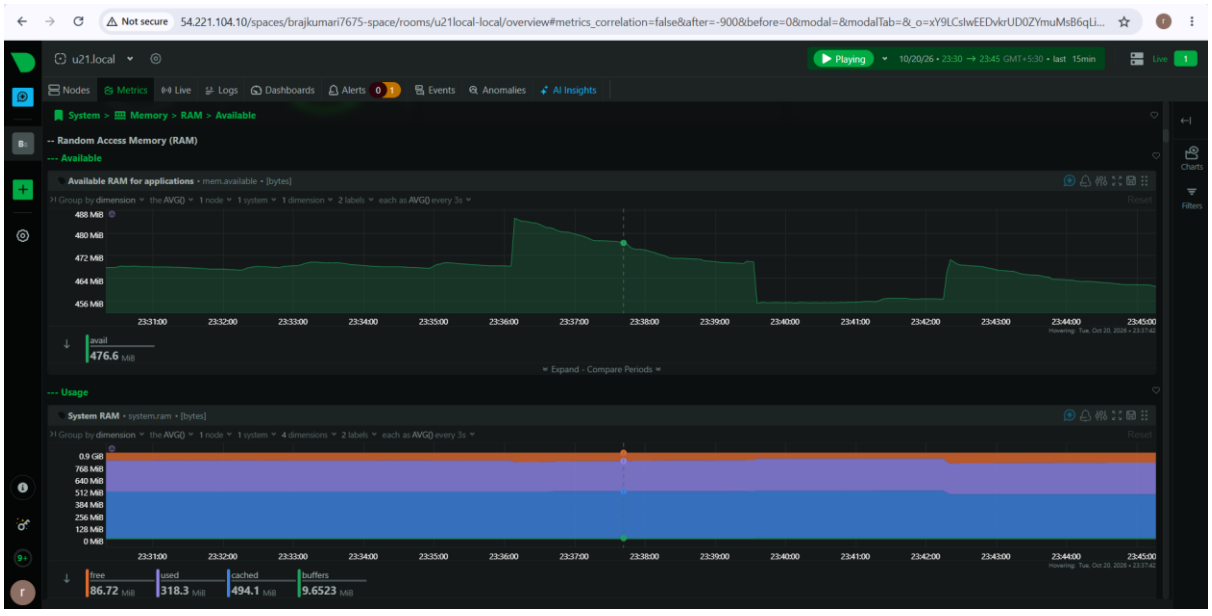

Not secure
54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/overview#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=fcyxCgIxDADQf8IsIGq5FtfbRRAnkZ...
u21.local
Paused
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
System > Compute > CPU
-- CPU
Total CPU utilization • system.cpu • [percent]
100 %
80 %
60 %
40 %
20 %
0 %
steal
0 %
softirq
0 %
user
0.5 %
system
0.8 %
iowait
0.2 %
Expand - Compare Periods
-- Pressure Stall Information (PSI)
--- CPU
---- Some Pressure
CPU some pressure • system.cpu_some_pressure • [percent]
CPU some pressure stall time • system.cpu_some_pressure_stall_time • [seconds]
10 ms
some 10
some 60
some 300
time


u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
System > Compute > PSI > Memory > Some
--- Memory
---- Some Pressure
Memory some pressure • system.memory_some_pressure • [percent]
Memory some pressure stall time • system.memory_some_pressure_stall_time • [seconds]
---- Full Pressure
Memory full pressure • system.memory_full_pressure • [percent]
Memory full pressure stall time • system.memory_full_pressure_stall_time • [seconds]
some 10
some 60
some 300
full 10
full 60
full 300
time
Expand - Compare Periods
26°C
Mostly clear
Search
ENG
IN

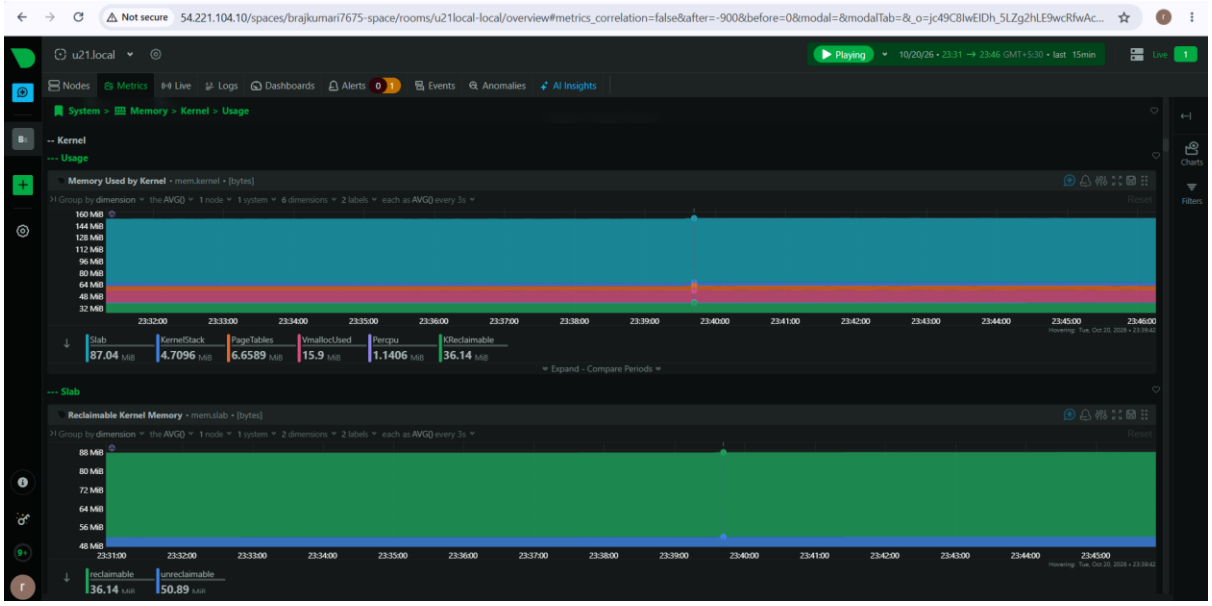

Not secure
54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/overview#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=zY_LCslwEEX_JWsHoi1tcCsl3UghS5...
u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
System > Compute > PSI > Storage > Some
---- Some Pressure
I/O some pressure • system.io_some_pressure • [percent]
I/O some pressure stall time • system.io_some_pressure_stall_time • [seconds]
---- Full Pressure
I/O full pressure • system.io_full_pressure • [percent]
I/O full pressure stall time • system.io_full_pressure_stall_time • [seconds]
some 10
some 60
some 300
full 10
full 60
full 300
time
Expand - Compare Periods
Charts
Filters

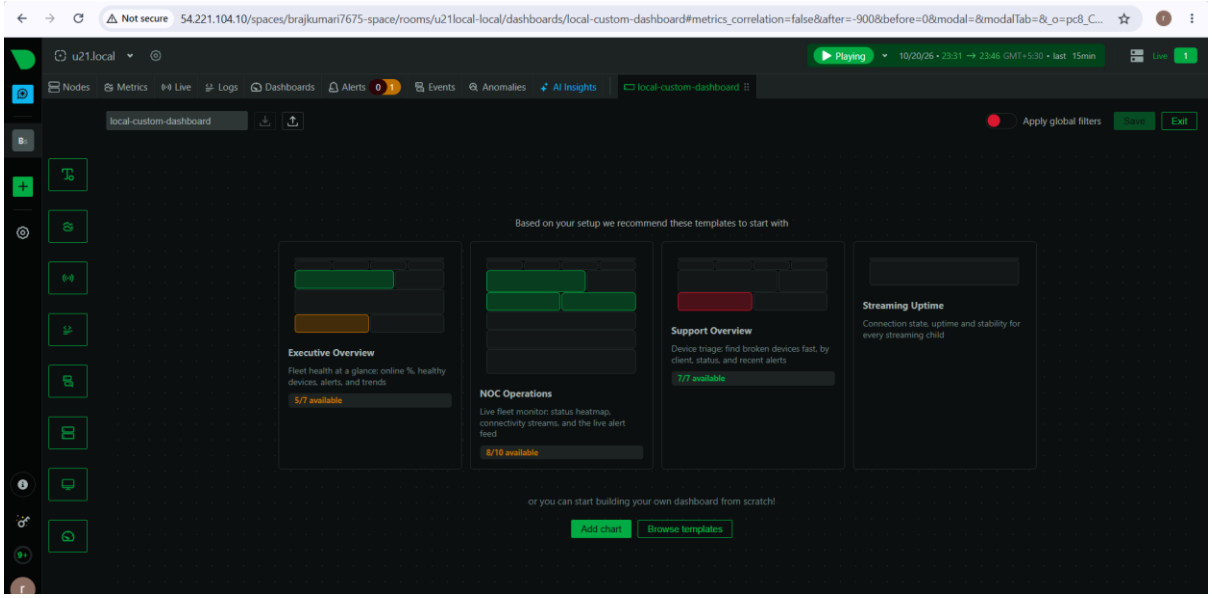

u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
System > Compute > Load
-- Load
System Load Average
load1
load5
load15
-- Clock
System Clock Status
unsync
clockerr

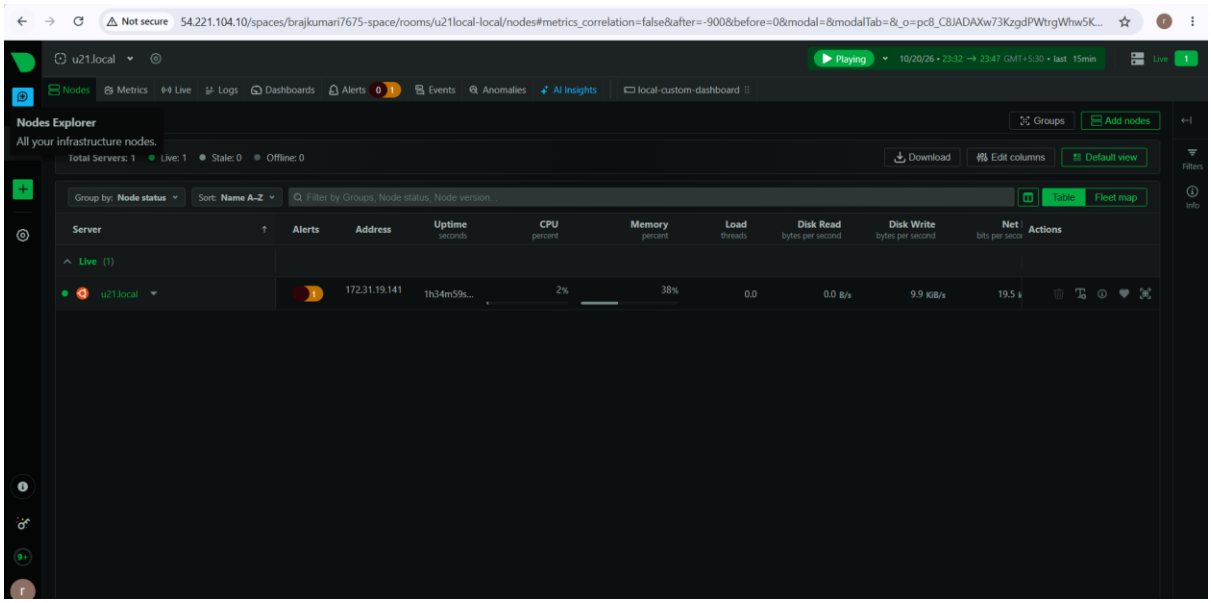

Not secure
54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/overview#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=rc5NCsIwEEDhu2TtwKihCW6LWzcu...
u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
System > Compute > Entropy
-- Entropy
Available Entropy
entropy
-- Linux Audit
Audit Backlog
used

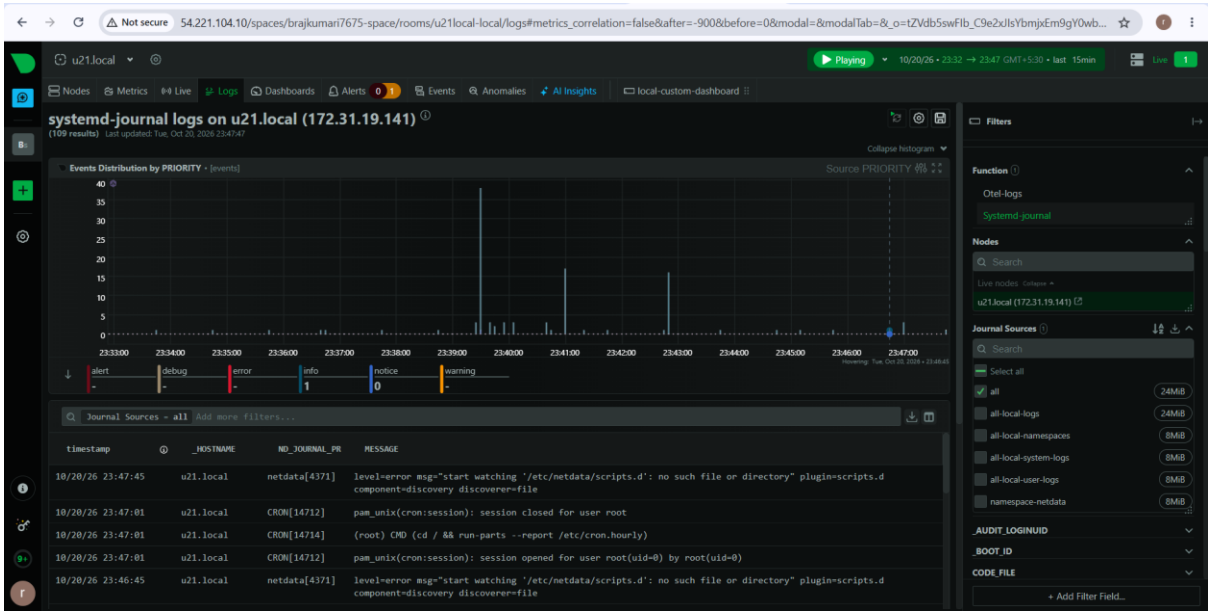

Not secure
54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/overview#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=xY9LCsIwEEDvkrUD0ZYmuMsB6qLi...
u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
System > Memory > RAM > Available
-- Random Access Memory (RAM)
--- Available
Available RAM for applications
avail
--- Usage
System RAM
free
used
cached
buffers

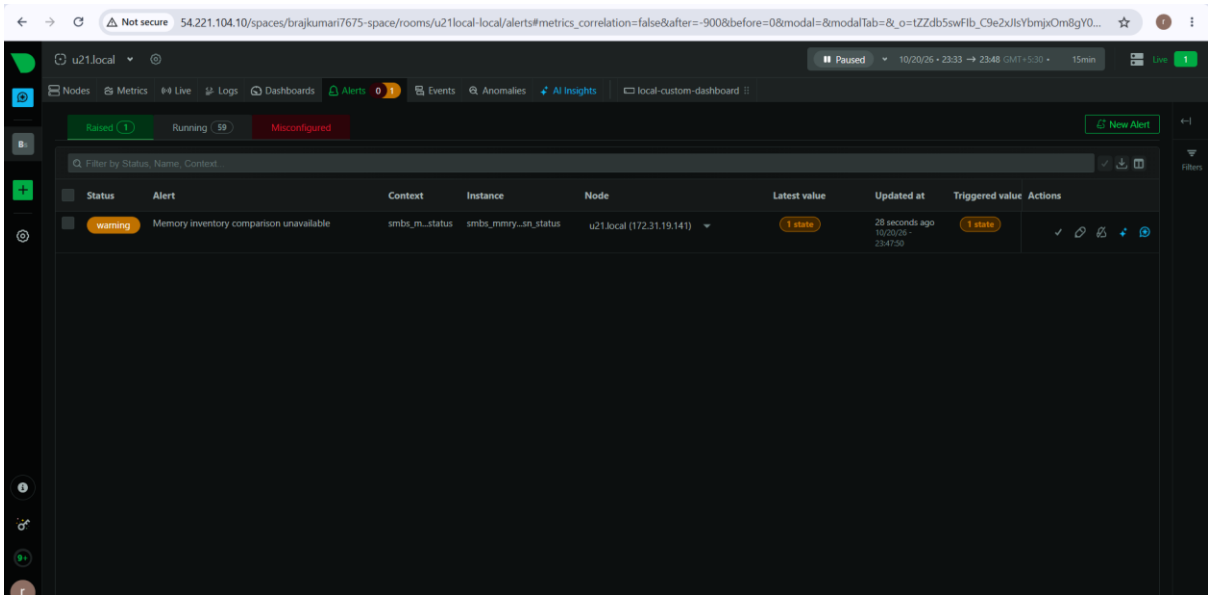

Not secure
54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/overview#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=tc6xCslwEADQf8nsQbSICW5-gA46...
u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
System > Memory > Activity > Reclaiming
Memory Reclaiming • mem.reclaiming • [bytes]
Active 242.4 MiB
Inactive 419.1 MiB
Active(anon) 145.8 MiB
Inactive(anon) 68.88 MiB
Active(file) 96.53 MiB
Inactive(file) 350.2 MiB
Unevictable 24.85 MiB
Mlocked 8.6055 MiB
--- Writeback
Writeback Memory • mem.writeback • [bytes]
Dirty 52 KiB
Writeback 0 B
--- Paging I/O

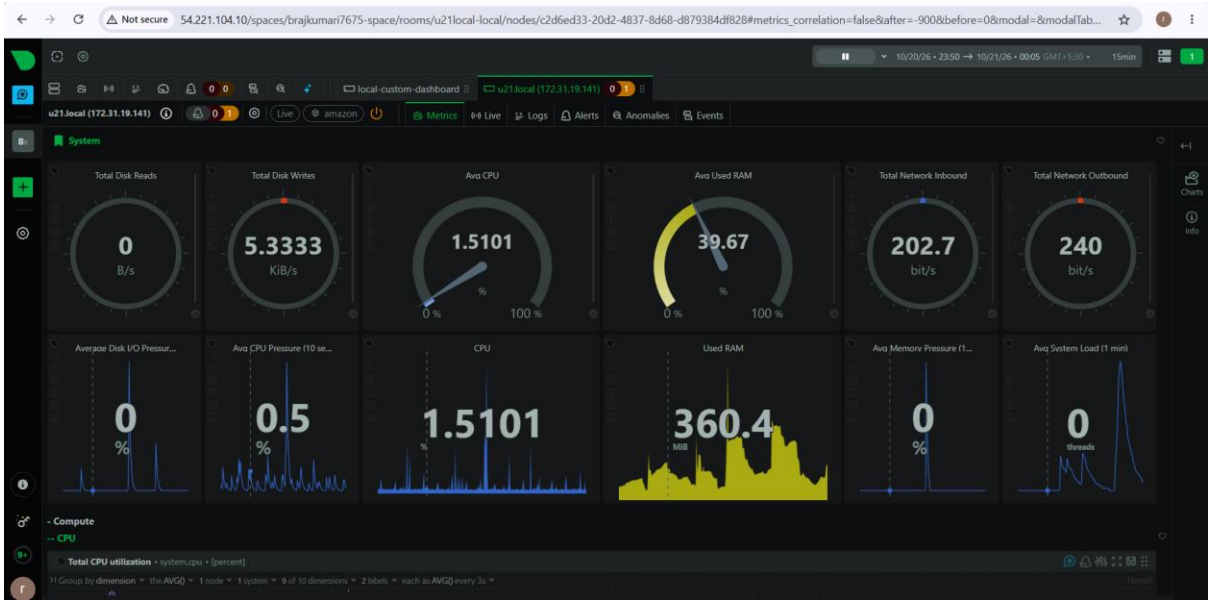

Not secure
54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/overview#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=jc49C8IwEIDh_5LZg2hLE9wcRfwAc...
u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
System > Memory > Kernel > Usage
-- Kernel
--- Usage
Memory Used by Kernel • mem.kernel • [bytes]
Slab 87.04 MiB
KernelStack 4.7096 MiB
PageTables 6.6589 MiB
VmallocUsed 15.9 MiB
Percpu 1.1406 MiB
KReclaimable 36.14 MiB
--- Slab
Reclaimable Kernel Memory • mem.slab • [bytes]
reclaimable 36.14 MiB
unreclaimable 50.89 MiB


Not secure
54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/dashboards/local-custom-dashboard#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=pc8_C...
u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
local-custom-dashboard
local-custom-dashboard
Apply global filters
Save
Exit
Based on your setup we recommend these templates to start with
Executive Overview
Fleet health at a glance: online %, healthy devices, alerts, and trends
5/7 available
NOC Operations
Live fleet monitor: status heatmap, connectivity streams, and the live alert feed
8/10 available
Support Overview
Device triage: find broken devices fast, by client, status, and recent alerts
7/7 available
Streaming Uptime
Connection state, uptime and stability for every streaming child
or you can start building your own dashboard from scratch!
Add chart
Browse templates


Not secure 54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/nodes#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=pc8_C8JADAXw73KzgdPWtrgWhw5K...
u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
local-custom-dashboard
Nodes Explorer
All your infrastructure nodes.
Total Servers: 1 Live: 1 Stale: 0 Offline: 0
Groups
Add nodes
Download
Edit columns
Default view
Table
Fleet map
Server
Alerts
Address
Uptime
CPU
Memory
Load
Disk Read
Disk Write
Net
Actions
Live (1)
u21.local
172.31.19.141
1h34m59s...
2%
38%
0.0
0.0 B/s
9.9 KiB/s
19.5 k


Not secure 54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/logs#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=tZVdb5swFIb_C9e2xJIsYbmjxEm9gY0wb...
u21.local
Playing
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
local-custom-dashboard
systemd-journal logs on u21.local (172.31.19.141)
(109 results)
Events Distribution by PRIORITY
alert
debug
error
info
notice
warning
Filters
Function
Otel-logs
Systemd-journal
Nodes
Search
u21.local (172.31.19.141)
Journal Sources
Select all
all
all-local-logs
all-local-namespaces
all-local-system-logs
all-local-user-logs
namespace-netdata
_AUDIT_LOGINUID
_BOOT_ID
CODE_FILE
+ Add Filter Field...
timestamp
_HOSTNAME
ND_JOURNAL_PR
MESSAGE
10/20/26 23:47:45 u21.local netdata[4371] level=error msg="start watching '/etc/netdata/scripts.d': no such file or directory" plugin=scripts.d component=discovery discoverer=file
10/20/26 23:47:01 u21.local CRON[14712] pam_unix(cron:session): session closed for user root
10/20/26 23:47:01 u21.local CRON[14714] (root) CMD (cd / && run-parts --report /etc/cron.hourly)
10/20/26 23:47:01 u21.local CRON[14712] pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
10/20/26 23:46:45 u21.local netdata[4371] level=error msg="start watching '/etc/netdata/scripts.d': no such file or directory" plugin=scripts.d component=discovery discoverer=file


Not secure 54.221.104.10/spaces/brajkumari7675-space/rooms/u21local-local/alerts#metrics_correlation=false&after=-900&before=0&modal=&modalTab=&_o=tZZdb5swFIb_C9e2xJIsYbmjxOm8gY0...
u21.local
Paused
Nodes
Metrics
Live
Logs
Dashboards
Alerts
Events
Anomalies
AI Insights
local-custom-dashboard
Raised 1
Running 59
Misconfigured
New Alert
Status
Alert
Context
Instance
Node
Latest value
Updated at
Triggered value
Actions
warning
Memory inventory comparison unavailable
smbs_m...status
smbs_mmry...sn_status
u21.local (172.31.19.141)
1 state
28 seconds ago
1 state



Final steps — verify the complete CI lab

1. Check Netdata on the backend

Run on your Ansible control node:

```
ansible backend -i ansible/inventory.ini -b \
  -m command -a "systemctl is-active netdata"
```

Expected output: active

2. Check Nginx on the frontend

```
ansible frontend -i ansible/inventory.ini -b \
  -m command -a "systemctl is-active nginx"
```

Expected output: active

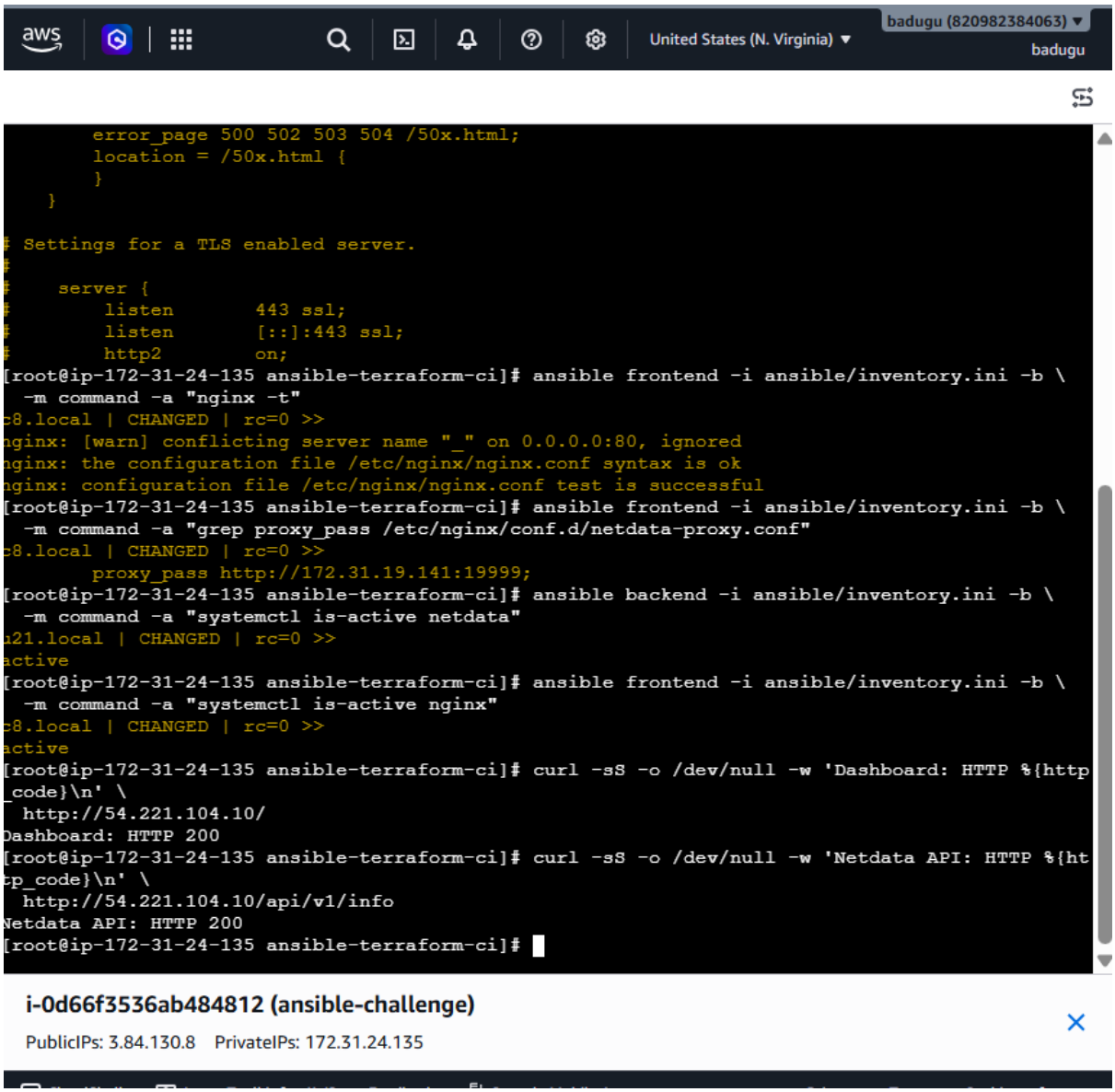
3. Test the dashboard through the reverse proxy

```
curl -sS -o /dev/null -w 'Dashboard: HTTP %{http_code}\n' \
http://54.221.104.10/
```

Then test the Netdata API:

```
curl -sS -o /dev/null -w 'Netdata API: HTTP %{http_code}\n' \
http://54.221.104.10/api/v1/info
```

Expected: HTTP 200 for both.



1. Make the configuration fully repeatable: ensure your playbook and templates use the backend's private IP.
2. Secure the lab: restrict backend port 19999 to the frontend security group; avoid exposing it publicly.
3. Add Terraform: provision both EC2 instances, security groups, and networking as code.
4. Build CI/CD: use Jenkins to validate your Ansible playbooks and run deployment steps automatically.

One important note: your earlier Nginx output showed a duplicate server-name warning. Since the endpoints work, it is not blocking the lab, but clean it up before treating this as production-ready.

